# Distributed control and cyber-physical systems design project

**PROJECT**

# Stima distribuita della posizione di un oggetto mediante rete di sensori ed intervento coordinato di droni.

**Keywords:** distributed algorithms, position estimate, esp8266, esp32, matlab, simulink

## Abstract

Il presente lavoro affronta il problema della stima distribuita della posizione di un oggetto mediante una rete di sensori. L'obiettivo è determinare la posizione di una sorgente rilevabile senza ricorrere a un nodo centralizzato, sfruttando esclusivamente la cooperazione locale tra i nodi della rete. Il sistema fisico è realizzato mediante quattro nodi ESP8266 interconnessi tramite il protocollo PainlessMesh, ciascuno equipaggiato con un sensore di prossimità HC-SR04. La scelta di tale tipologia di sensore è il risultato di un processo di selezione che ha escluso approcci alternativi basati su sensori di temperatura e infrarossi, rivelatisi non praticabili a causa di limitazioni fisiche e ambientali discusse nel seguito. Ogni nodo rileva la propria distanza dall'oggetto e, attraverso un algoritmo di consenso distribuito in cui ciascun nodo comunica esclusivamente con i due vicini adiacenti, la rete converge a una stima condivisa della posizione mediante multilaterazione. Inoltre, per correggere eventuali errori, a ciò segue una fase di media distribuita. L'approccio è stato preliminarmente validato in ambiente Simulink, dapprima in condizioni ideali e successivamente introducendo le principali non idealità del sistema reale: oggetto non puntiforme, errore di misura dei sensori, ritardi di comunicazione e il packet loss della rete. I risultati mostrano la robustezza dell'algoritmo proposto rispetto a tali perturbazioni e ne confermano l'applicabilità in scenari di sensing distribuito reale. Fatto ciò, la stima di posizione è usata per implementare l'intervento di due droni, di cui uno solo conosce tale stima di posizione, perciò sarà necessario un algoritmo del consenso con leader per far si che la destinazione sia nota ad ogni drone.

## 1. Analisi delle soluzioni di sensing candidate e motivazione della scelta

L'obiettivo di tale progetto è determinare la posizione in maniera distribuita partendo dalla distanza rilevata da un sensore. Vari sensori sono stati considerati e contestualizzati in vari scenari.

### 1.1. Rilevamento di un incendio

L'idea iniziale era quella di realizzare un sistema capace di determinare, in maniera distribuita la posizione di un incendio mediante una rete distribuita di sensori di temperatura **ds18b20**.




**Figure 1.** ds18b20.

Possiamo dividere il problema in **due parti**:

1. Ogni nodo, letta la temperatura misurata dal sensore, calcola la distanza da esso dell'incendio.
2. Eseguire l'algoritmo distribuito per determinarne la posizione.

Il secondo punto non è un problema, infatti, nel seguito sarà realizzato, piuttosto il primo step, ovvero convertire la temperatura rilevata in posizione, presenta delle complicazioni:

- La variazione di temperatura rispetto a quella ambientale non dipende solo dalla distanza della fiamma dal sensore, ma anche dalla sua dimensione, ciò significa che per

determinare la posizione dell'incendio è necessario conoscere a priori la sua dimensione, un vero e proprio controsenso.
    - Assumiamo di conoscere a priori tale dimensione
- La dimensione della fiamma non è costante, perciò quello che inizialmente è un incendio a X cm potrebbe diventare con il passar del tempo un incendio a 2X cm.
    - Assumiamo che la fiamma possa essere considerata di dimensioni costanti.
- Anche se la dimensione della fiamma è nota a priori e costante nel tempo, il sensore non è così sensibile da poter permettere una effettiva conversione C°->cm. Variazioni significative ed effettivamente utilizzabili le si hanno solo se la fiamma è estremamente vicina, limitando quindi il progetto.

A causa di queste problematiche tale sensore è stato escluso. L'alternativa, sempre rimanendo nel tema della rilevazione di incendi, è un sensore ad infrarossi **ky-026**, con il quale è possibile rilevare una fiamma, poiché emette luce IR.



**Figure 2.** ky-026.

Anche in questo caso vi sono **due step**:

1. Ogni nodo, letta l'intensità di luce IR misurata dal sensore, calcola la distanza da esso dell'incendio,
2. Eseguire l'algoritmo distribuito per determinarne la posizione.

Il secondo step, come detto prima, è in seguito sviluppato. Il problema rimane sempre il primo step:

- Sono principalmente usati come sensori on/off, perciò hanno un'uscita digitale alta se una fiamma è stata rilevata. Nonostante ciò, hanno anche un'uscita analogica inversamente proporzionale all'intensità di luce IR.
- Tale variazione è molto più graduale e precisa rispetto alla variazione di temperatura, ma ciò dipende ancora dalla dimensione della fiamma. Perciò il sensore rileverebbe la stessa intensità di una fiamma piccola ma vicina per una fiamma grande ma lontana.
    - Assumiamo come prima di poter considerare la fiamma di dimensione nota e costante.
- Il valore misurato è influenzato anche dallo sfarfallio della fiamma.

Il valore di luce IR, letto dal sensore mediante pin analogico dell'esp, varia tra 0 e 1023 (1023 intensità IR minima, 0 intensità IR massima). Tale valore, purtroppo anche se più utilizzabile rispetto alla temperatura, risulta ancora troppo instabile, lasciando una fiamma sempre nella stessa posizione, quanto letto oscilla troppo. Ad esempio, a 8cm del sensore, limitando al minimo lo sfarfallio, il valore va da 600 a 900, rendendo troppo incerto il calcolo della distanza da usare per applicare l'algoritmo di stima della posizione distribuita.

Poiché il nostro obiettivo è quello di sviluppare un algoritmo di stima della posizione a partire dalla distanza rilevata dal sensore, ma la distanza, calcolabile a partire dai valori di temperatura o intensità IR misurati, soffre di incertezza troppo elevata, l'idea di rilevare un incendio mediante tale tipo di sensori, senza dover ricorrere a sensori più complessi e costosi come telecamere termiche, risulta essere irrealizzabile.

### 1.2. Rilevamento di un intruso

**Mutiamo quindi leggermente l'idea iniziale**, ovviamente l'obiettivo principale resta sempre il determinare in maniera distribuita la posizione di qualcosa a partire dalla distanza di questo rispetto ai nodi. L'idea è quindi rilevare la presenza di un **intruso**, misurarne la distanza

dal nodo mediante **sensori di prossimità** (ad ultrasuoni) **hc-sr04**, e utilizzare tale valore come già detto prima per applicare l'algoritmo distribuito.




**Figure 3.** hc-sr04.

Il funzionamento di tali sensori non è particolarmente complicato, per ottenere una misura e necessario sollecitare con un impulso alto per 10µs il pin **TRIGGER**. Il sensore, quindi, emetterà un'onda e alzerà il segnale sul pin **ECHO**, l'onda rimbalzerà sull'oggetto e tornerà al sensore, il quale quando la capta abbassa il segnale sul pin **ECHO**. Misurato per quanto tempo il segnale echo resta alto, ovvero il tempo che tale onda ha impiegato per arrivare all'oggetto e tornare al sensore, si può calcolare la distanza.

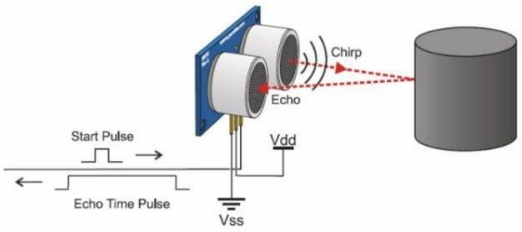



**Figure 4.** Funzionamento hc-sr04.

La distanza dell'oggetto la si ottiene mediante la seguente formula:

$$cm = \frac{durata * 0{,}034}{2} \quad (1)$$

Dove **0,034 cm/µs** è la velocità del suono e il fratto 2 è necessario poiché ricordiamo che la durata è il tempo complessivo di andata e ritorno, quindi per percorrere due volte la distanza. Leggiamo dal datasheet:

- Power Supply: +5V DC
- Quiescent Current: <2mA
- Working current: 15mA
- Effectual Angle: <15º
- Ranging Distance: 2 400 cm
- Resolution: 0.3 cm
- Measuring Angle: 30º
- Trigger Input Pulse width: 10uS
- Dimension: 45mm x 20mm x 15mm
- Weight: approx. 10 g

Vediamo che la **risoluzione** dichiarata è di 0.3cm, verificata anche sperimentalmente, la misura è molto più precisa e stabile rispetto a quanto ideato prima, quindi perfetto per il nostro obiettivo e come input dell'algoritmo. Un'altra caratteristica importate è **l'angolo di visuale** di 30°, fondamentale nel seguito.

## 2. Progettazione del sistema

Scelto il tipo di sensore da usare per determinare la distanza dell'oggetto (intruso) dal nodo, progettiamo l'intero sistema.

### 2.1. Posizionamento dei nodi

La prima cosa da determinare è dove tali nodi saranno posizionati, per fare ciò consideriamo in primis una delle caratteristiche evidenziate nel datasheet, l'angolo di visuale: **30°**:

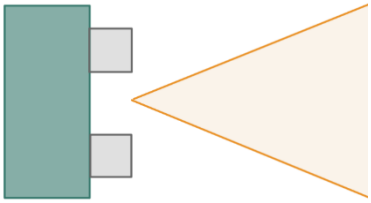


**Figure 5.** Angolo di visione del sensore.

Consideriamo ora di avere 4 sensori (nodi) disposti nel seguente modo:

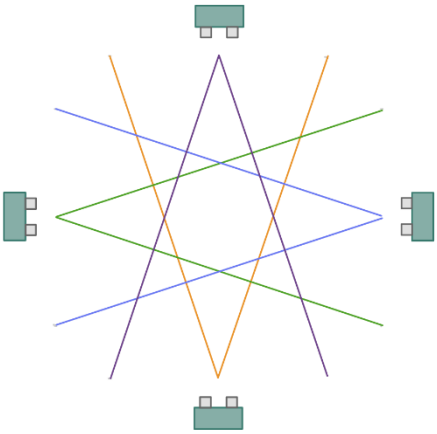


**Figure 6.** Disposizione di 4 nodi.

Le visuali si sovrappongono permettendo quindi di effettuare una multiangolazione. Ma con tali nodi che area ci conviene "sorvegliare".

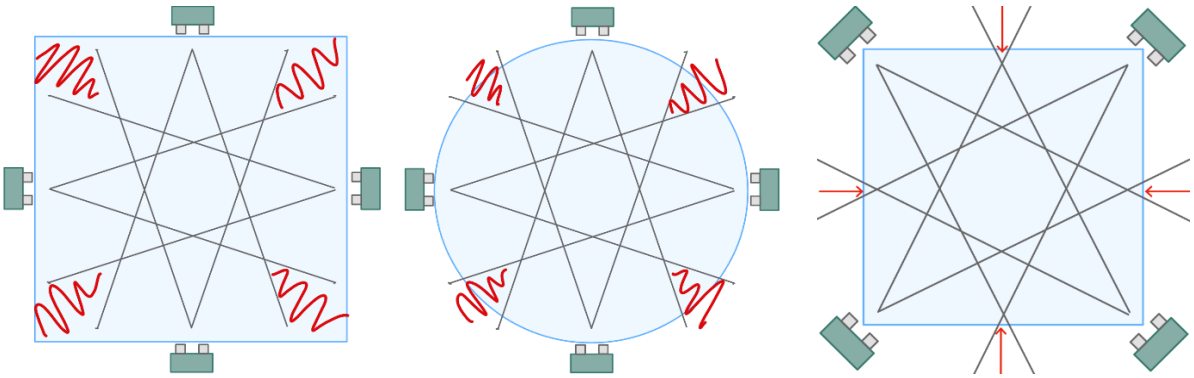


**Figure 7.** Aree non coperte.

Vediamo quindi che ci sono delle **aree non coperte**, ovviamente, non potrà essere rilevato l'intruso. Scegliamo la terza configurazione ma mostriamo una altra cosa importantissima, vi sono anche delle aree coperte da solo uno o due sensori, perciò l'intruso viene rilevato, ma la posizione non può essere determinata poiché è impossibile effettuare una triangolazione/multiangolazione, come intuibile anche dal nome della tecnica.

Nella pratica ciò può essere migliorato utilizzando più sensori e/o sensori con un angolo di visione maggiore.

### 2.2. Determinazione della posizione

Scelto il numero di nodi, la posizione e l'area da supervisionare, mostriamo come le loro misure, ovvero la distanza dell'oggetto da ogni nodo, possano essere utilizzata per **determinare la posizione dell'intruso**. Consideriamo quindi un quadrato di lato 17, i sensori saranno posizionati negli angoli a coordinate: (0,0), (0,17), (17,0) e (17,17).

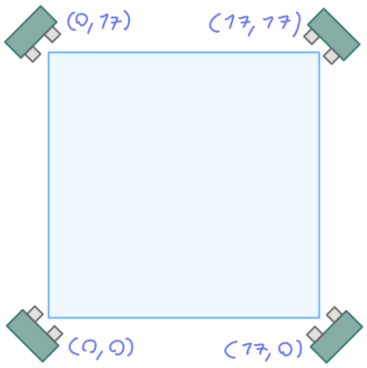



**Figure 8.**

Assumiamo ora la presenza di un intruso in una regione coperta.

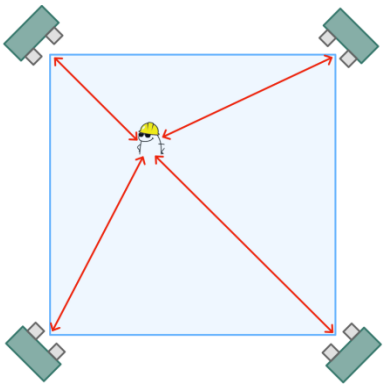


**Figure 9.** Presenza di un intruso.

La misura di distanza fornita dai sensori rappresenta il **raggio di una circonferenza** centrata nel nodo, su cui può trovarsi l'intruso.

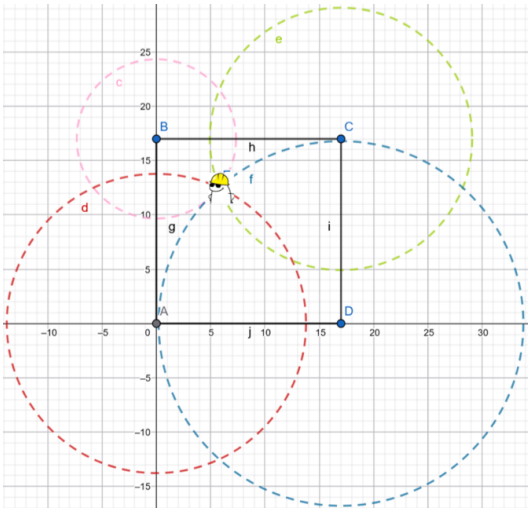



**Figure 10.** Intersezione delle circonferenze.

Quindi per determinare la posizione dell'intruso è sufficiente risolvere un **sistema di equazioni**, formato dalle 4 equazioni delle circonferenze:

$$\begin{cases} (x-0)^2 + (y-0)^2 = d1^2 \\ (x-0)^2 + (y-17)^2 = d2^2 \\ (x-17)^2 + (y-0)^2 = d3^2 \\ (x-17)^2 + (y-17)^2 = d4^2 \end{cases} \quad (2)$$

Risolvendolo troviamo quindi l'intersezione delle 4 circonferenze, non che posizione dell'intruso. Si fa notare che tale sistema è sovra dimensionato. Purtroppo, quanto mostrato è molto **ideale**, in quanto stiamo considerando un oggetto puntiforme e che le distanze fornite dai sensori siano estremamente precise e non affette da rumore. Nella realtà allora potremmo assumere che tale punto si trovi in un'area delimitata dall'intersezione di 4 corone circolari.

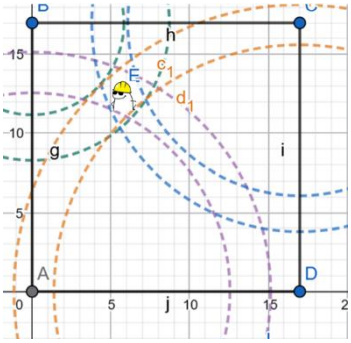



**Figure 11.** Situazione più realistica

Perciò se provassimo a risolvere tale sistema non otterremmo nessun risultato, nessuna intersezione. Esistono quindi vari metodi matematici per la risoluzione approssimata di tali sistemi. Non approfondiamo ciò poiché la risoluzione (precisa o approssimata) di tali sistemi richiede la conoscenza delle misure di ogni sensore, ciò porta quindi ad un approccio **centralizzato**. Ogni nodo, quindi, invia la propria distanza rilevata ad un nodo master il quale risolve il sistema di equazioni. Analogamente se ogni nodo ricevesse le distanze misurate dagli altri e risolvesse il sistema sarebbe sempre considerato centralizzato.

### 2.3. Stima distribuita della posizione

Mostriamo ora come sia possibile determinare la posizione dell'intruso in maniera **distribuita**. Ogni nodo esegue il medesimo algoritmo, il quale prevede che scambiando solo con i propri vicini, non la distanza misurata, ma la stima di posizione, ogni nodo possa corregge la propria fino a che non convergano allo stesso valore. Scegliamo quindi la seguente topologia in cui ogni nodo parla con solo due suoi vicini:

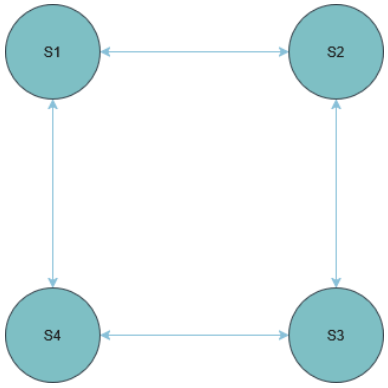
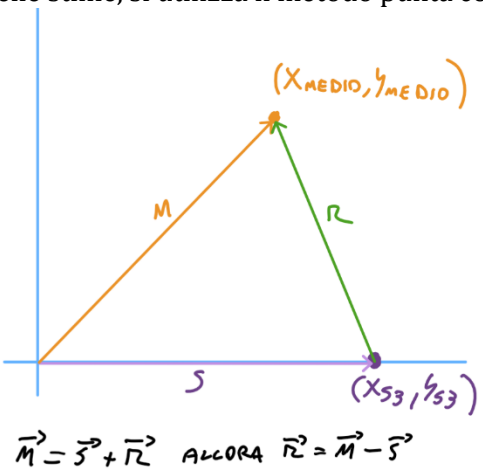



**Figure 12.** Topologia

$$A = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}; D = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix} \quad (3)$$

Tale topologia può essere descritta in maniera equivalente con un grafo o in forma matriciale, dove A è la matrice di incidenza e D è la matrice diagonale del grado di ogni nodo (ovvero il numero di nodi con cui comunica). Detto ciò, definiamo i seguenti valori per poter introdurre l'algoritmo utilizzato e il suo pseudocodice:

- $X_i(k)$ *e* $Y_i(k)$ rappresentano la stima della posizione dell'intruso del nodo i-esimo all'istante k.
- $X_{medio} = \frac{X_i(k)+\sum_{j\in N_i} a_{ij}X_J}{(1+d_i)}$; $Y_{medio} = \frac{Y_i(k)+\sum_{j\in N_i} a_{ij}Y_J}{(1+d_i)}$
-
    - Per entrambe le coordinate si calcola la media tra la stima mantenuta dal nodo i e le stime ricevute dai vicini, ovvero $X_J$ (e $Y_J$).
    - $N_i$ rappresenta l'insieme di tutti i $d_i$ vicini del nodo i-esimo.
    - Si fa fratto $1 + d_i$ per la media aritmetica poiché si somma alla propria stima quella dei $d_i$ vicini.
- $rx = X_{medio} - X_{sensore_i}$ $\quad ry = Y_{medio} - Y_{sensore_i}$
    - Si calcola il vettore che va dal sensore al punto che ha come coordinate la media delle stime, si utilizza il metodo punta coda:




    - Tale vettore lo si calcola per determinare la distanza tra il sensore e l'attuale punto stimato.
- Perciò calcoliamo la lunghezza del vettore non che distanza tra sensore e punto stimato $r = \sqrt{rx^2 + ry^2}$
- $error = r - Distanza_{misurata}$ calcolando la differenza tra la distanza tra sensore e punto stimato e effettiva distanza misurata dal sensore possiamo determinare l'errore commesso. Se l'errore è positivo il punto stimato è ancora troppo lontano dal nodo, viceversa è troppo vicino.
- $u_x = \frac{rx}{r}$; $\quad u_y = \frac{ry}{r}$
    - Capito di quanto stiamo sbagliando è necessario capire come spostare l'attuale stima; quindi, si calcola $u_x$ per determinare come spostare la stima di x e $u_y$ per determinare come spostare la stima di y.

Definite tali grandezze, **l'algoritmo** è il seguente:

$$X_i(k+1) = X_{medio} - \alpha * error * u_x$$
$$Y_i(k+1) = Y_{medio} - \alpha * error * u_y \quad (4)$$

Effettuata la media tra la propria posizione stimata e quella dei vicini la si corregge andando a sottrarre un termine formato da tre aliquote:

- $\alpha$ passo di aggiornamento, permette di rendere l'aggiornamento della stima più o meno significativo.
- $error$ che rende il fattore di correzione più o meno grande in base a quanto stiamo sbagliando e il verso.
- $u_x$ perché appunto potremmo star sbagliando anche a causa di una sola delle due coordinate o comunque a causa maggiormente di una.

Vediamo sotto forma di **pseudo codice** come ciò può essere implementato in ogni nodo:

```
Inizializza comune della variabile di stima.
Leggi misura di sensore di prossimità.

Se distanza < soglia allora tale nodo vede l'intruso, quindi, invia
la propria stima ai vicini.
Xmedio= media tra mia stima e stima ricevuta dai vicini di
coordinata x
Ymedio= media tra mia stima e stima ricevuta dai vicini di
coordinata y
rx = x_medio-x;
ry = y_medio-y;
r=sqrt(rx*rx+ry*ry);
error=r-cm;
ux=rx/r;
uy=ry/r;
x_stimato = Xmedio - 0.1 * error * ux;
y_stimato = Ymedio - 0.1 * error * uy;

Viceversa, se distanza>soglia, significa che tale nodo non vede
l'intruso, perché non presente nel capo visivo, perciò, non invia
nulla ai vicini, ma media solamente le stime ricevute dai vicini in
modo che possa essere anche lui consapevole della posizione
dell'intruso.
```

## 3. Simulazione in matlab/simulink

Prima di realizzare fisicamente tutto ciò è necessario **simularlo**, prima in forma **ideale** poi aggiungendo alcune **non idealità** come:

- Oggetto non puntiforme
- Rumore sovrapposto alla lettura del sensore
- Disturbi provocati dalla rete, quali delay e packet loss.

### 3.1. Modello del processo

Il primo step è quindi modellare il processo fisico, un **oggetto fisico che riflette onde** (ultrasuoni) le quali impiegheranno un tempo differente ad arrivare al nodo, in base a dove si trova tale oggetto. Tale modello è stato realizzato mediante la seguente matlab function:

```
function [sx_up, sx_low, dx_up, dx_low]= fcn(x_r, y_r)

if x_r==-1 && y_r==-1
        sx_up=22;
        sx_low=22;
        dx_low=22;
        dx_up=22;
else
```

```
    %sensore in (0,0)
    %0=a*0+q->q=0
    %y_r=a*x_r+0

    a=y_r/x_r;
    %vedo se è nel range di coefficienti angolari per
    %cui il punto è nel cono di visione
    if a>tand(30) && a<tand(60)
        sx_low=sqrt((0-x_r)^2+(0-y_r)^2);
    else
        sx_low=22;
    end

    %sensore in (17,17)
    %17=a*17+q->q=17-a*17
    %y_r=a*x_r+q=a*x_r+17-a*17

    a=(y_r-17)/(x_r-17);
    %vedo se è nel range di coefficienti angolari per
    %cui il punto è nel cono di visione
    if a>tand(30) && a<tand(60)
        dx_up=sqrt((17-x_r)^2+(17-y_r)^2);
    else
        dx_up=22;
    end

    %sensore in (0,17)
    %17=a*0+q->q=17
    %y_r=a*x_r+q=a*x_r+17

    a=(y_r-17)/(x_r);
    %vedo se è nel range di coefficienti angolari per
    %cui il punto è nel cono di visione
    if a>tand(120) && a<tand(150)
        sx_up=sqrt((0-x_r)^2+(17-y_r)^2);
    else
        sx_up=22;
    end

    %sensore in (17,0)
    %0=a*17+q->q=-a17
    %y_r=a*x_r+q=a*x_r-a17

    a=(y_r)/(x_r-17);
    %vedo se è nel range di coefficienti angolari per
    %cui il punto è nel cono di visione
    if a>tand(120) && a<tand(150)
        dx_low=sqrt((17-x_r)^2+(0-y_r)^2);
    else
        dx_low=22;
    end
end

end
```

Tale funzione in base alla posizione dell'oggetto ricevuta in input restituisce il valore di distanza da ogni nodo. Infatti, le uscite [sx_up, sx_low, dx_up, dx_low] rappresentano proprio il valore di distanza rilevato dal nodo in alto a sinistra, in basso a sinistra etc. Se non vi è nessuno oggetto, ovvero le coordinate in ingresso sono (-1,-1) la funzione restituisce la distanza massima, ovvero 22, poiché essendo un quadrato di lato 17, la diagonale è 24, per prevenzione limitiamo la misura

massima a 22. Viceversa, se effettivamente c'è un oggetto restituisce ad un nodo la distanza da esso se e solo se l'oggetto è nel campo di visione, ciò è determinabile calcolando il coefficiente angolare della retta passante per l'oggetto e il nodo in questione. Se tale coefficiente è compreso tra i coefficienti delle due rette che delimitano superiormente ed inferiormente il campo di visione, allora significa che il nodo lo vede, perciò viene calcolata la distanza (come distanza di due punti nel piano) e la si restituisce. Infine, ultimo caso, è presente un oggetto, quindi coordinate diverse da -1, ma non è nel campo di visione di un nodo, allora come nel caso di assenza dell'oggetto si restituisce la distanza massima.
Perciò l'architettura che modella il processo è la seguente:

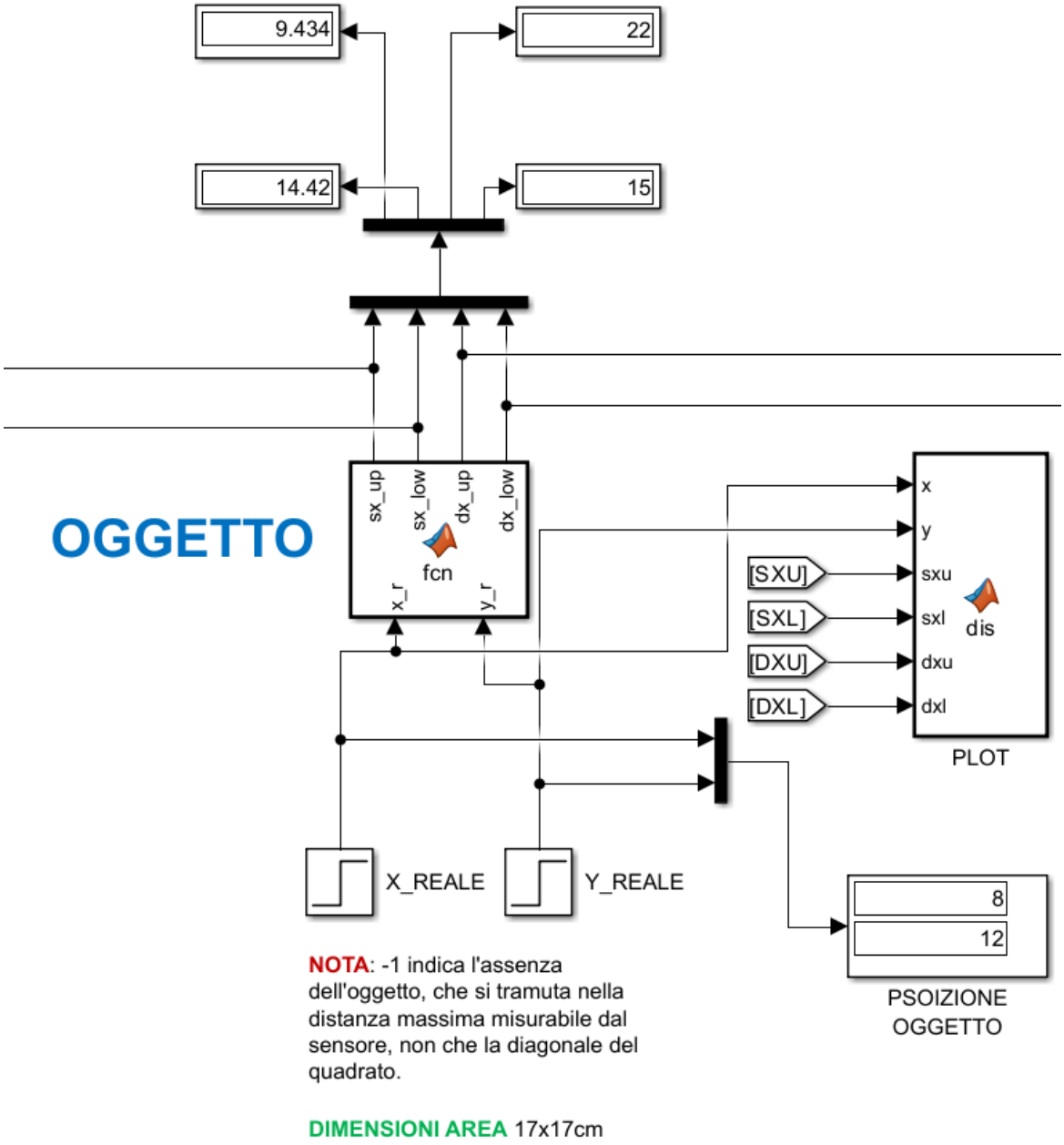



**Figure 13.** Modello del processo

Il blocco centrale è la matlab function sopra presentata, vi sono due blocchi step per emulare l'inserimento di un oggetto, un display che mostra le coordinate reali, dei display per mostrare la distanza reale rispetto i nodi. Infine, il blocco **PLOT** serve per stampare la posizione effettiva dell'oggetto e le stime nel tempo dei nodi, ad esempio:

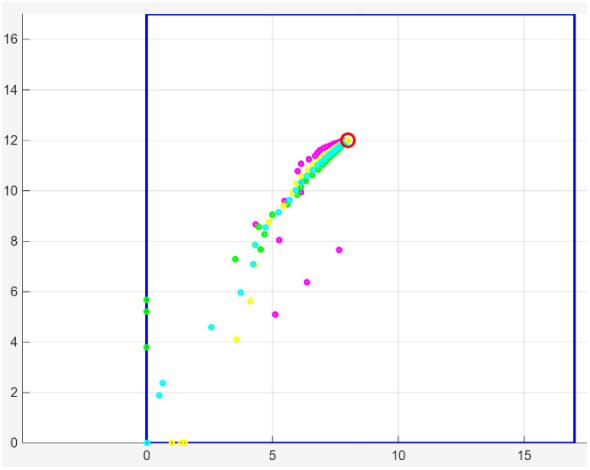



**Figure 14.** Esempio di plot

### 3.2. Modello del nodo

Modelliamo ora il **nodo**, il quale in questa **prima simulazione ideale** riceve esattamente il valore di distanza fornito dal modello del processo. Grazie a tale valore il nodo calcola la stima della posizione dell'oggetto utilizzando l'algoritmo presentato nei paragrafi precedenti.

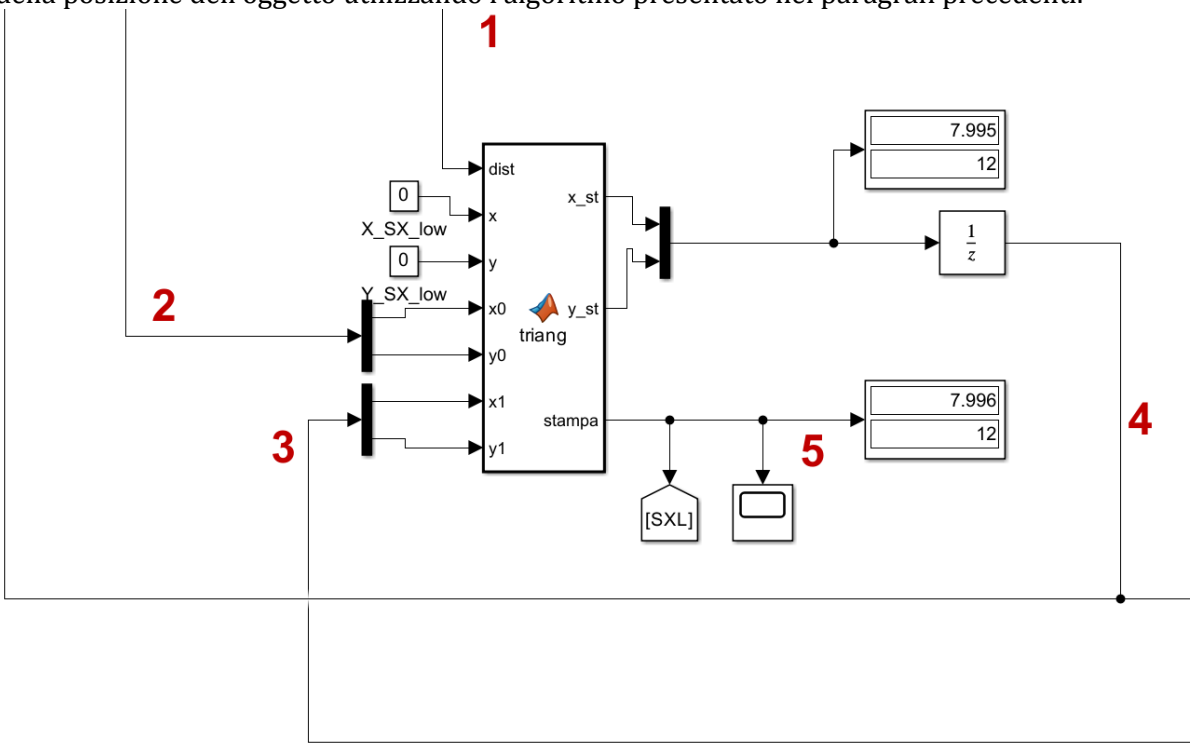



**Figure 15.** Modello del nodo

1. Rappresenta il valore di distanza proveniente direttamente e senza errori dal modello.
2. Stima di posizione di un vicino
3. Stima di posizione dell'altro vicino
4. Stima di posizione del nodo fornita ai nodi vicini solo se il nodo come detto vede l'oggetto.
5. Stima di posizione fornita sempre in output solo per poterla stampare.

Mostriamo quindi la funzione presente nel blocco matlab function, la quale rappresenta una implementazione matlab dell'algoritmo presentato nei paragrafi precedenti.

```
function [x_st, y_st,stampa] = triang(dist,x,y, x0, y0,x1,y1)
    persistent xst
```

```
    persistent yst;
    if isempty(xst) && isempty(yst)
        % xst=8.5;
        % yst=8.5;
        xst=0.1;
        yst=0.1;
    end

    if dist<=20
        x_st=xst;
        y_st=yst;
        lo_vedo=1;
    else
        x_st=-1;
        y_st=-1;
        lo_vedo=0;
    end

    x_medio=xst;
    y_medio=yst;
    n=1;

    if x0~=-1
        x_medio=x_medio+x0;
        y_medio=y_medio+y0;
        n=n+1;
    end
    if x1~=-1
        x_medio=x_medio+x1;
        y_medio=y_medio+y1;
        n=n+1;
    end

    x_medio=x_medio/n;
    y_medio=y_medio/n;

    if lo_vedo==1
        rx = x_medio-x;
        ry = y_medio-y;
        r=sqrt(rx*rx+ry*ry);
        error=r-dist;

        ux=rx/r;
        uy=ry/r;

        xst = x_medio - 0.5 * error * ux;
        yst = y_medio - 0.5 * error * uy;
    else
        xst = x_medio;
        yst = y_medio;
    end

    stampa=[xst,yst];

end
```

xst e yst rappresentano il valore di stima al passo k mantenuto da tale nodo, i quali vengono forniti in output solo se la misura di distanza è inferiore ad una data soglia, viceversa fornisce in output -1 per indicare che non sta comunicando. Perciò, se il nodo rileva un oggetto l'algoritmo eseguito è quello presentato analiticamente, ma come visto anche nello pseudocodice nel caso in cui il nodo non vede l'oggetto ma i vicini mandano delle stime di

posizioni diverse da -1 (-1 indica che il vicino non vede nulla) allora significa che c'è un oggetto ma non nel suo campo visivo, per far si che anche tale nodo sappia dove si trova l'intruso effettua la media tra la sua stima e quelle ricevute.

### 3.3. Architettura ideale complessiva

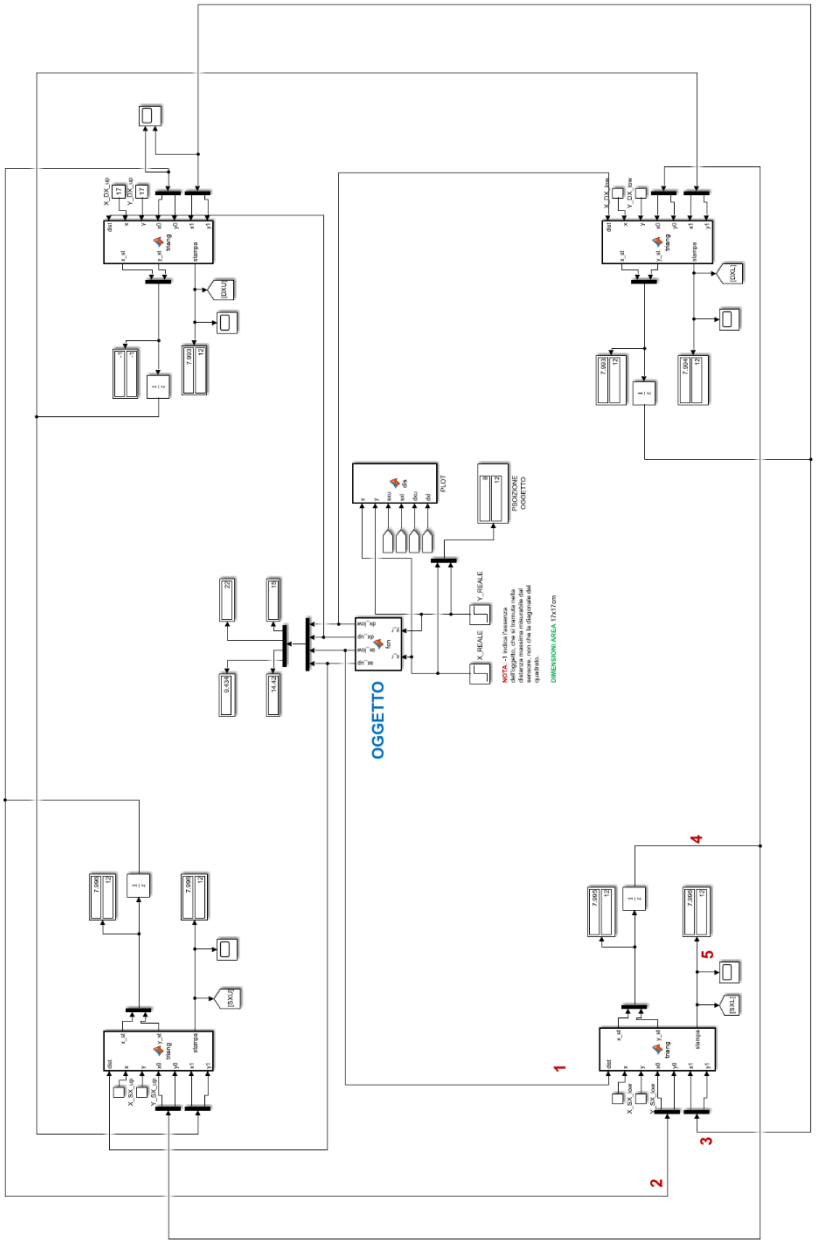



**Figure 16.** Architettura ideale complessiva

### 3.4. Simulazioni

Di seguito riportiamo i risultati di alcune simulazioni.

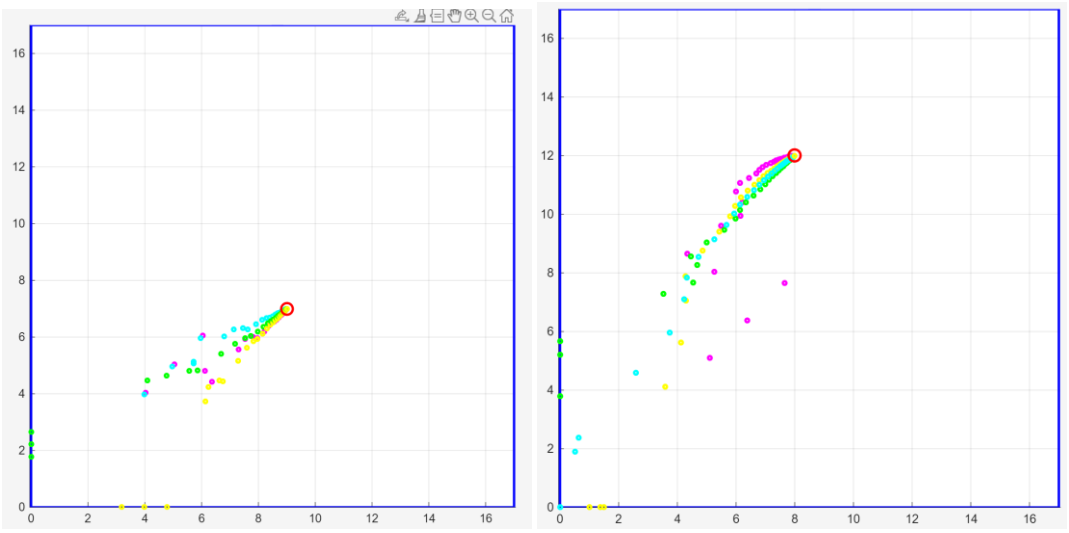


**Figure 17.** Simulazioni con successo

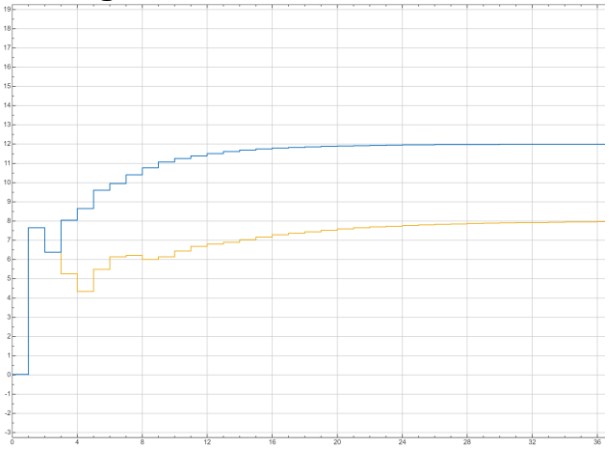


**Figure 18.** Conoscenza di un nodo (giallo:X, blu:Y)

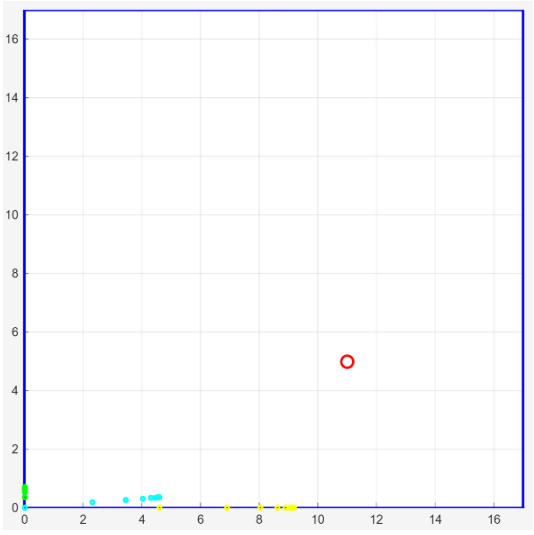


**Figure 19.** Simulazione con insuccesso

Come mostrato in fig. 19 non è stato possibile determinare la posizione dell'oggetto a causa di un numero non sufficiente di nodi che lo vedessero.

### 3.5. Non idealità

Evolviamo l'architettura prima presentata aggiungendo delle **non idealità**. In primis modifichiamo il modello del processo, il quale non fornirà più in output la distanza ma il **tempo** che l'onda ha impiegato per andare e tornare dall'oggetto. Perciò la matlab function del processo è la seguente:

```
function [sx_up, sx_low, debug, dx_up, dx_low]= fcn(x_r, y_r)

idle_dist=22;
idle_time=(idle_dist*2)/0.034;

if x_r==-1 && y_r==-1
        sx_up=idle_time;
        sx_low=idle_time;
        dx_low=idle_time;
        dx_up=idle_time;
        debug=[idle_dist,idle_dist,idle_dist,idle_dist];
else

    debug=[-1,-1,-1,-1];
    %sensore in (0,0)
    %0=a*0+q->q=0
    %y_r=a*x_r+0

    a=y_r/x_r;
    %vedo se è nel range di coefficienti angolari per
    %cui il punto è nel cono di visione
    if a>tand(30) && a<tand(60)
        cm=sqrt((0-x_r)^2+(0-y_r)^2);
        sx_low=(cm*2)/0.034;
        debug(2)=cm;
    else
        sx_low=idle_time;
        debug(2)=idle_dist;
    end

    %sensore in (17,17)
    %17=a*17+q->q=17-a*17
    %y_r=a*x_r+q=a*x_r+17-a*17

    a=(y_r-17)/(x_r-17);
    %vedo se è nel range di coefficienti angolari per
    %cui il punto è nel cono di visione
    if a>tand(30) && a<tand(60)
        cm=sqrt((17-x_r)^2+(17-y_r)^2);
        dx_up=(cm*2)/0.034;
        debug(3)=cm;
    else
        dx_up=idle_time;
        debug(3)=idle_dist;
    end

    %sensore in (0,17)
    %17=a*0+q->q=17
    %y_r=a*x_r+q=a*x_r+17

    a=(y_r-17)/(x_r);
    %vedo se è nel range di coefficienti angolari per
    %cui il punto è nel cono di visione

    if a>tand(120) && a<tand(150)
        cm=sqrt((0-x_r)^2+(17-y_r)^2);
```

```
        sx_up=(cm*2)/0.034;
        debug(1)=cm;
    else
        sx_up=idle_time;
        debug(1)=idle_dist;
    end

    %sensore in (17,0)
    %0=a*17+q->q=-a17
    %y_r=a*x_r+q=a*x_r-a17

    a=(y_r)/(x_r-17);
    %vedo se è nel range di coefficienti angolari per
    %cui il punto è nel cono di visione
    if a>tand(120) && a<tand(150)
        cm=sqrt((17-x_r)^2+(0-y_r)^2);
        dx_low=(cm*2)/0.034;
        debug(4)=cm;
    else
        dx_low=idle_time;
        debug(4)=idle_dist;
    end

end

end
```

La logica è la stessa di prima, banalmente se l'oggetto è nel campo di visione, calcolata la distanza, usa la formula inversa di (1) per fornire il tempo impiegato dall'onda.

Modifichiamo ora l'architettura in modo da porter simulare una misura ogni secondo, ricorda ottieni una misura ogni volta che stimoli il trigger del sensore, il quale risponde con echo da cui determini il tempo impiegato dall'onda. Perciò assumiamo (come fatto anche nell'implementazione fisica) di effettuare una misura ogni secondo. Inoltre, aggiungiamo del rumore sommando al valore ricevuto un numero randomico (non in cm ma in us). Infine, è stato sottratto al valore un tempo pari ad una distanza di 0,5 cm per emulare un oggetto non puntiforme, quel 29.4 nella figura seguente.

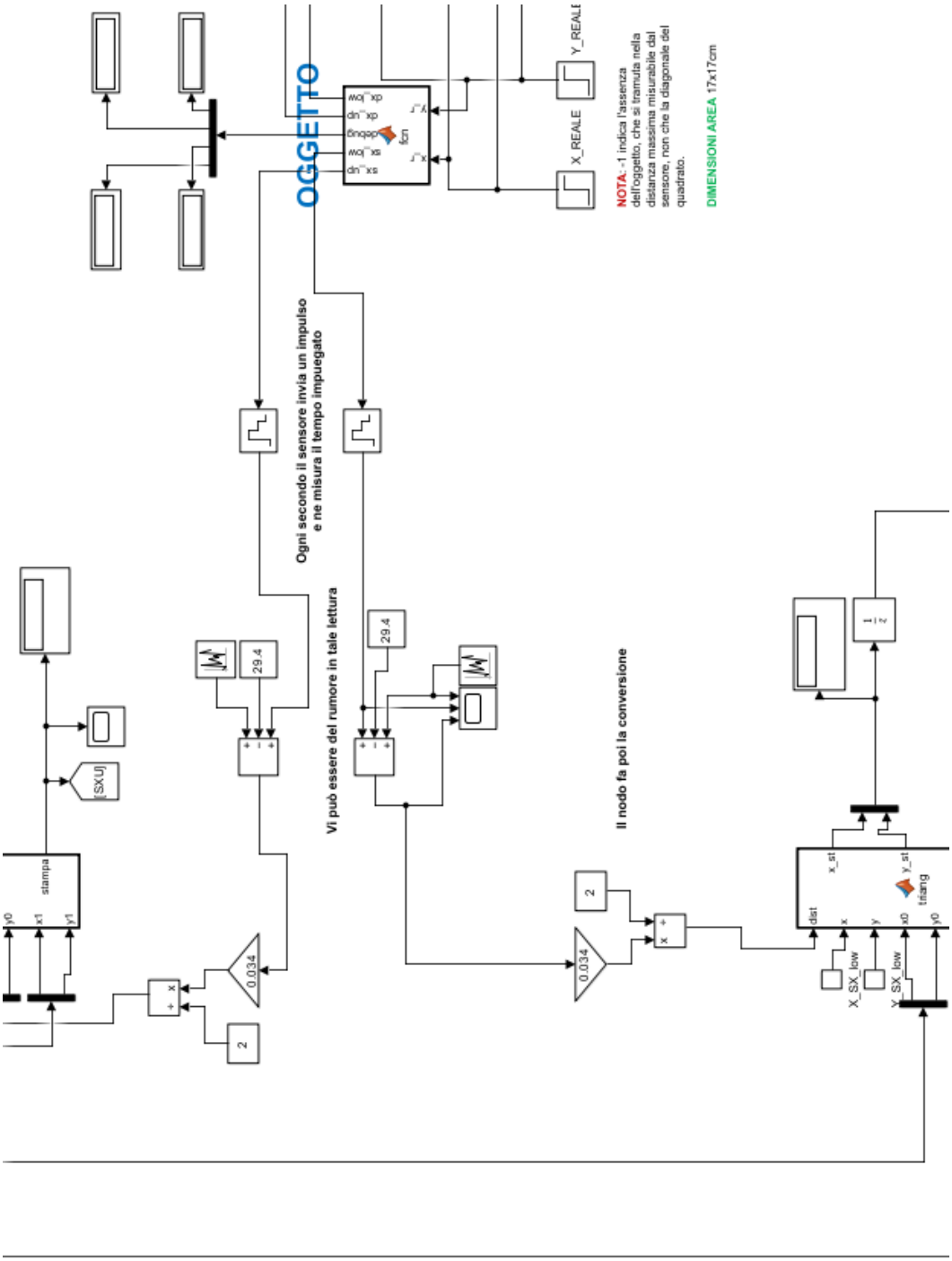



**Figure 20.** Architettura con non idealità

Come si nota da tale immagine, cosi come accade nei nodi reali, misurato l'intervallo di tempo in cui echo è alto, lo si converte in cm, usiamo quindi un blocco moltiplicatore e uno divisore in modo da non dover cambiare la matlab function del nodo e fornirgli in ingresso sempre il valore di distanza.

Riportiamo di seguito la nuova architettura complessiva:

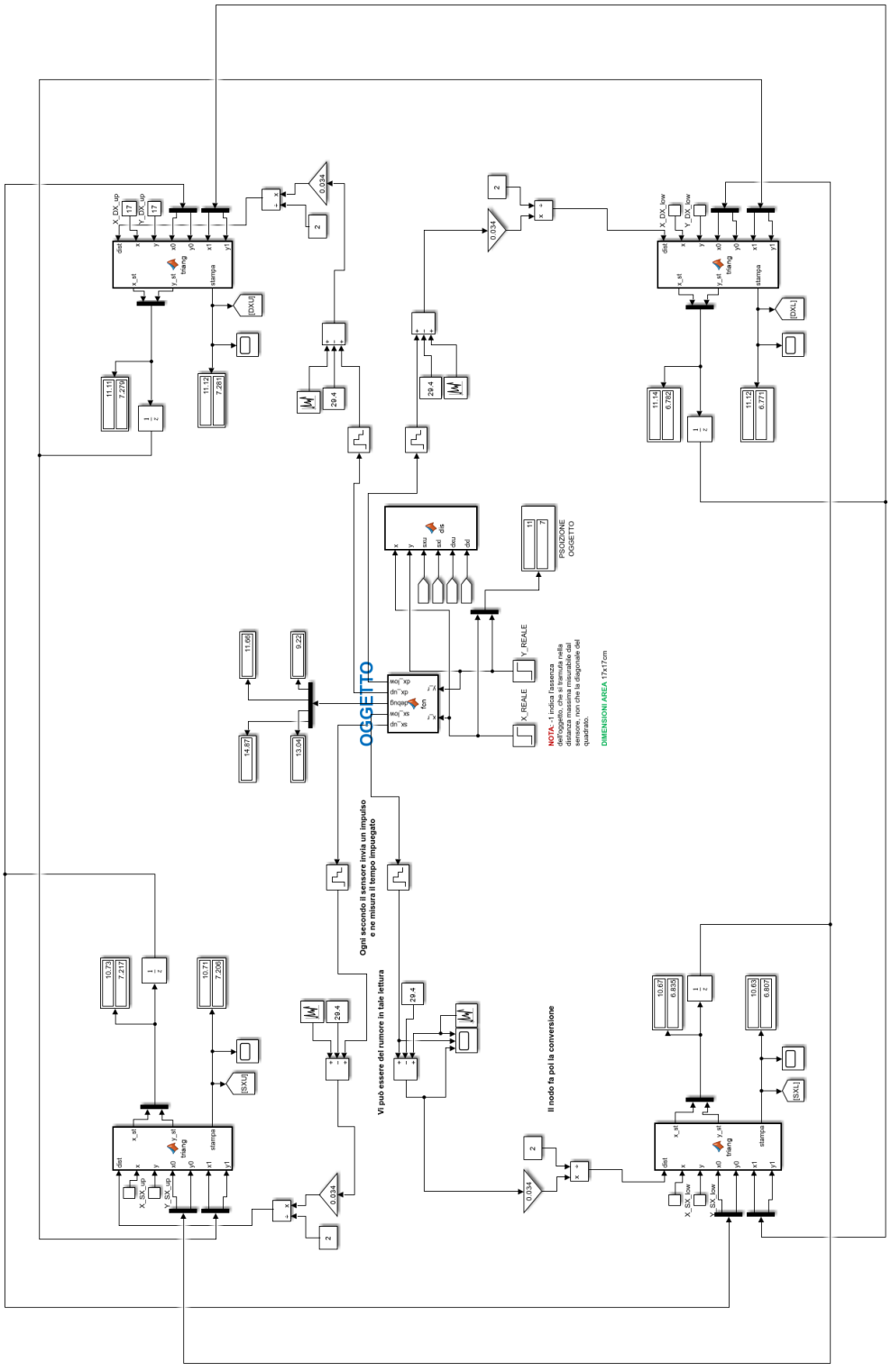



**Figure 21.** Architettura complessiva con non idealità

### 3.6. Simulazioni

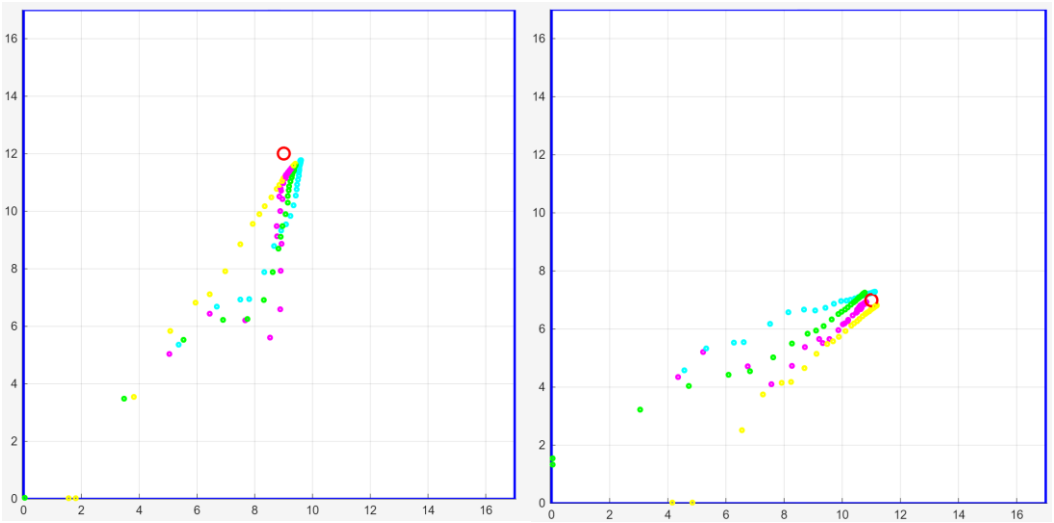



**Figure 22.** Simulazioni in presenza di non idealità

Come mostrato a causa delle non idealità non si converge esattamente al punto ma si nota una cosa, se facessimo uno zoom sulla seconda simulazione:

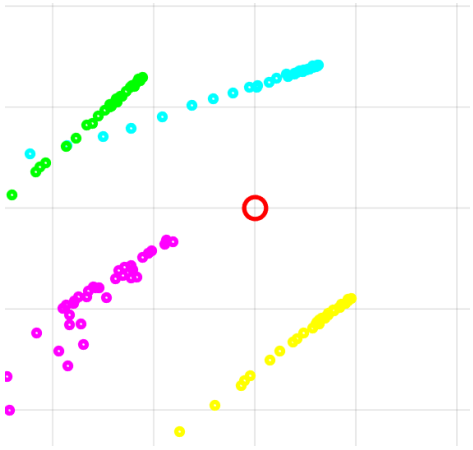


**Figure 23.** Vista di dettaglio nel grafico.

Si nota perfettamente l'effetto dell'oggetto non puntiforme, infatti ogni nodo pensa che l'oggetto sia più vicino ad esso. **Vedremo nel seguito come migliorare ciò aggiungendo un ulteriore fase di media distribuita**.

#### 3.7. Network

Aggiungiamo infine l'ultima non idealità, ovvero la presenza di una **rete non perfetta**, la quale introduce ritardi nelle comunicazioni e/o perdite di pacchetti. Per fare ciò utilizziamo il modello di NET visto a lezione, il quale ha come ingressi ed uscite:

- Banda (rate in uscita)
- Rate in ingresso
- Dati in ingresso
- Dati in uscita

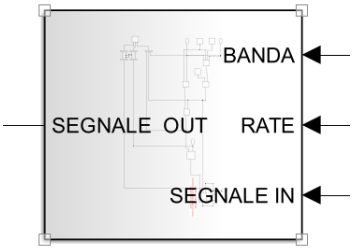



**Figure 24.** Sottosistema della rete.

Perciò scegliamo una banda, non sovradimensionata, per modellare uno scenario realistico, una rate casuale per simulare varie condizioni di carico, ed anche eventuali attacchi e sovraccarichi della rete. In segnale IN forniamo l'output del nodo, ovvero la stima di x e di y che il nodo vuole fornire ai suoi vicini. Segnale OUT invece sarà collegato all'ingresso dei nodi a cui quest'ultimo vuole effettivamente comunicare. Perciò usiamo quattro di questi sottosistemi, a cui rispettivamente forniamo in ingresso la stima di uno dei nodi, e l'uscita è collegata a due vicini. L'architettura complessiva risulta essere:

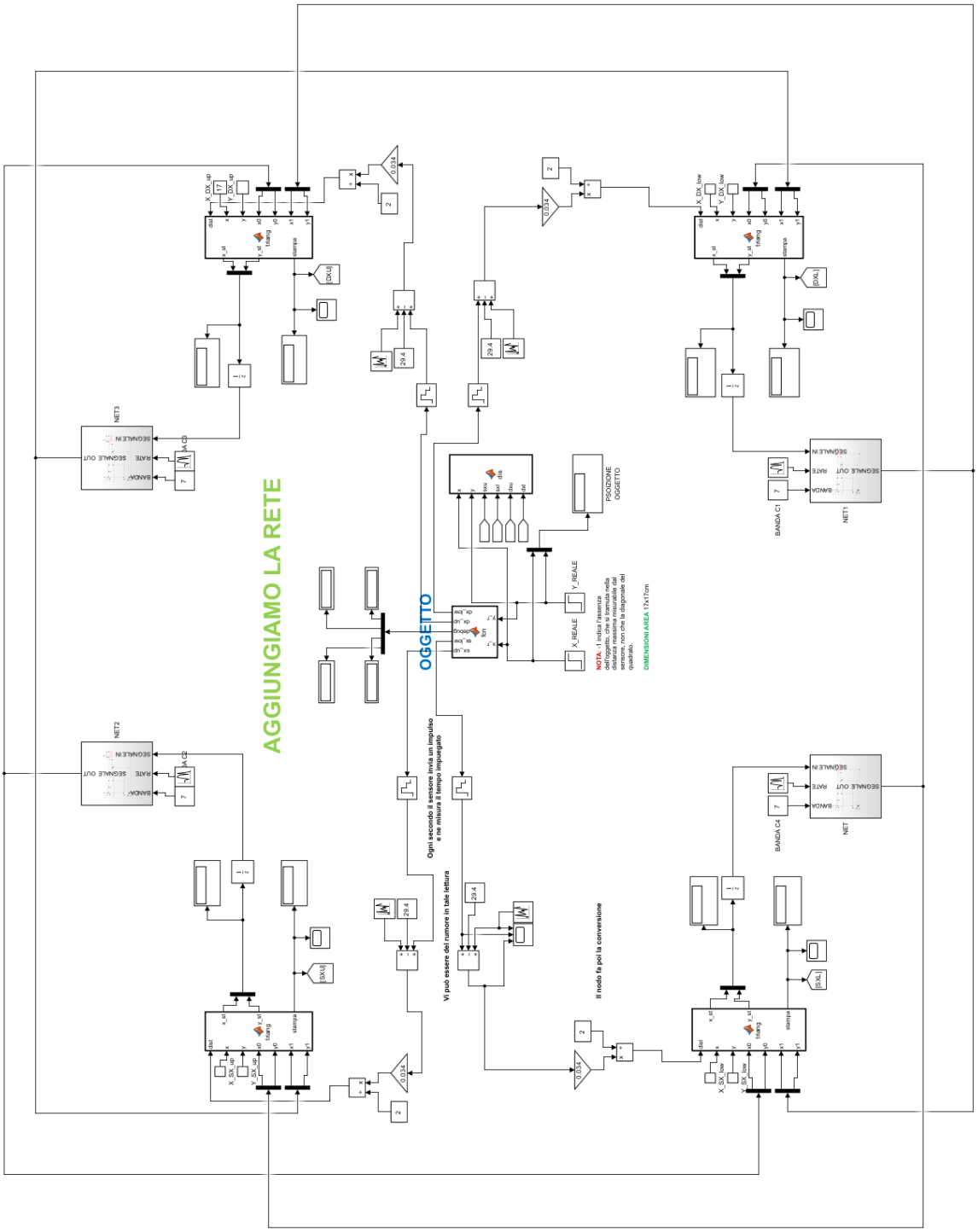



**Figure 25.** Architettura con rete

### 3.8. Simulazione

Possiamo simulare ora l'intero sistema considerando sia le precedenti non idealità sia la rete. Riportiamo il risultato di alcune simulazioni e l'effetto della rete sui messaggi di un nodo.

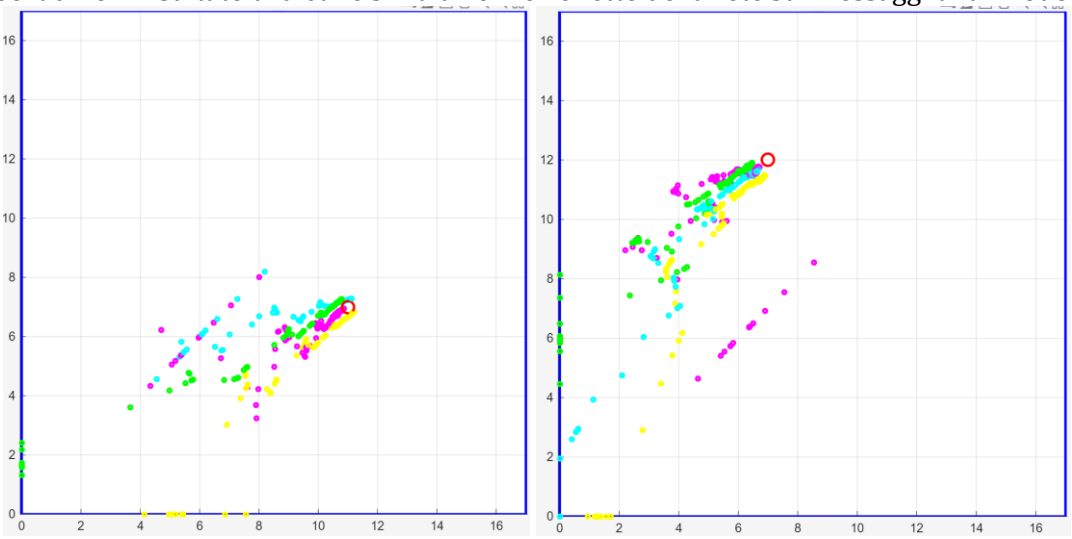


**Figure 26.** Simulazioni

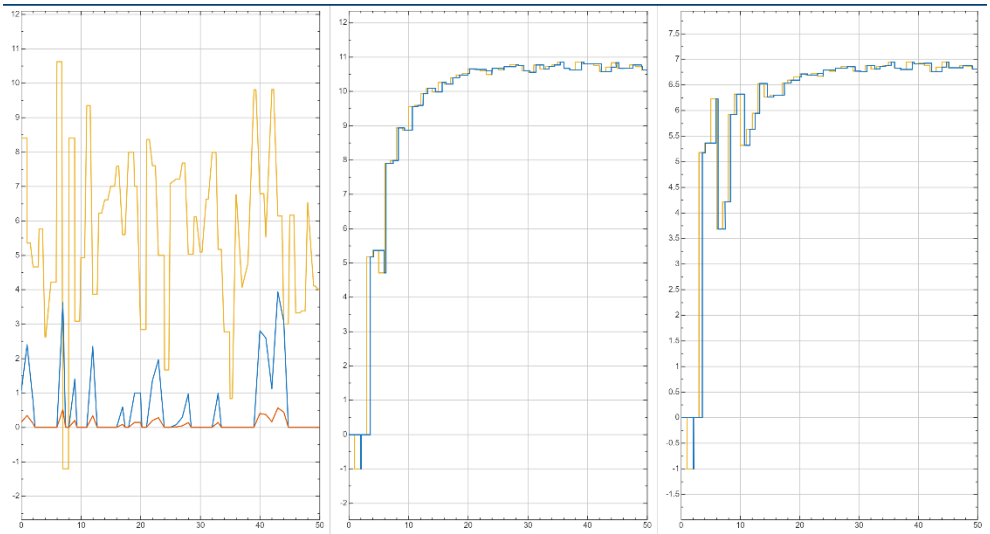


**Figure 27.** Effetto della rete (a)

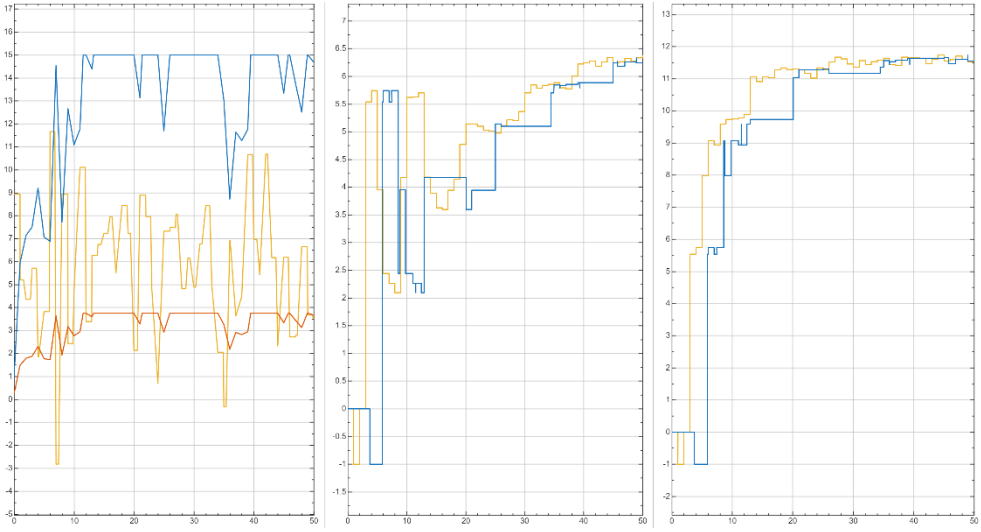


**Figure 27.** Effetto della rete (b)

In fig.26 vediamo che nonostante la presenza della rete, insieme alle precedenti non idealità, la rete converge. Inoltre, in fig.27 possiamo vedere:

- 1° grafico da sinistra: in blu il livello della rete, in giallo il rate in ingresso, in rosso il ritardo
- 2° grafico: in giallo il segnale originario, in blu il segnale ritardato e con effetto di perdita di pacchetto in uscita. (coordinata X)
- 3° grafico: analogo al grafico 2 ma per la coordinata Y.

In figura 27b notiamo un rate in ingresso molto elevato che comporta un elevato ritardo e perdita di pacchetto; infatti, il segnale in uscita rimane piatto in alcuni intervalli. Nonostante ciò, se guardassimo il valore di stima, a cui convergono i nodi che utilizzano tali dati, è ancora corretto:

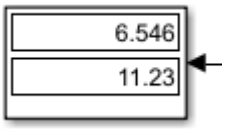



**Figure 28.** Stima di un nodo

### 3.9. Introduzione della media distribuita

Capito che il nostro sistema funziona correttamente anche in presenza di non idealità vogliamo **migliorarlo**. Come notato nelle varie simulazioni, i vari nodi convergono piu o meno allo stesso valore, con un errore piu che accettabile, soprattutto se si ricorda che l'oggetto non è puntiforme, ricordiamo la seguente figura che lo evidenziava:

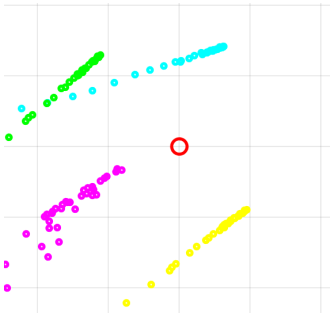


**Figure 29.**

Detto ciò, possiamo migliorare l'algoritmo in modo tale che ogni nodo converga alla media delle stime ottenute, cosi che tutti decidano esattamente per lo stesso punto. Per fare ciò modifichiamo l'algoritmo, dopo un certo numeri di passi/iterazioni, il nodo smette di eseguire l'algoritmo di stima di posizione ed esegua l'algoritmo di media distribuita visto a lezione. Più precisamente usiamo un algoritmo basato sul consenso tempo discreto per effettuare la media dei valori di stima dei nodi ad un certo istante di tempo:

$$\begin{gathered} X_i(k+1) = X_i(k) - \varepsilon \sum_{j \in N_j} a_{ij} \left(X_i - X_j\right) \\ X_i(0) = xst \\ Y_i(k+1) = Y_i(k) - \varepsilon \sum_{j \in N_j} a_{ij} \left(Y_i - Y_j\right) \\ Y_i(0) = yst \end{gathered} \tag{5}$$

Ovvero il valore di partenza usato dal nodo per applicare tale algoritmo è l'ultimo valore di stima calcolato al passo precedente dall'altro algoritmo.

Inoltre $\varepsilon$, passo di discretizzazione, deve essere compreso tra 0 e $\frac{1}{d_M}$ dove $d_M$ è il grado massimo, nel nostro esempio 2, perciò $\varepsilon = \frac{1}{2} = 0.5$.

La matlab function che modella il nodo è così modificata:

```
function [x_st, y_st,stampa] = triang(dist,x,y, x0, y0,x1,y1,t)
```

```
persistent xst
persistent yst;

if isempty(xst) && isempty(yst)
    % xst=8.5;
    % yst=8.5;
    xst=0.1;
    yst=0.1;
end


if t<=40
    if dist<=20
        x_st=xst;
        y_st=yst;
        lo_vedo=1;
    else
        x_st=-1;
        y_st=-1;
        lo_vedo=0;
    end

    x_medio=xst;
    y_medio=yst;
    n=1;

    if x0~=-1
        x_medio=x_medio+x0;
        y_medio=y_medio+y0;
        n=n+1;
    end
    if x1~=-1
        x_medio=x_medio+x1;
        y_medio=y_medio+y1;
        n=n+1;
    end

    x_medio=x_medio/n;
    y_medio=y_medio/n;

    if lo_vedo==1
        rx = x_medio-x;
        ry = y_medio-y;
        r=sqrt(rx*rx+ry*ry);
        error=r-dist;

        ux=rx/r;
        uy=ry/r;

        xst = x_medio - 0.5 * error * ux;
        yst = y_medio - 0.5 * error * uy;
    else
        xst = x_medio;
        yst = y_medio;
    end

    stampa=[xst,yst];
else
```

```
        if x0~=-1
            xst=xst-0.5*1*(xst-x0);
            yst=yst-0.5*1*(yst-y0);
        end
        if x1~=-1
            xst=xst-0.5*1*(xst-x1);
            yst=yst-0.5*1*(yst-y1);
        end

        x_st=xst;
        y_st=yst;

        stampa=[xst,yst];
    end


end
```

Ogni nodo ha in ingresso un riferimento temporale, in base ad esso inizialmente effettua la stima di posizione, ma da un certo istante in poi (nell'esempio da 40 in poi) effettua la media distribuita mediante l'algoritmo basato sul consenso mostrato prima. Ciò può essere notato mostrando l'andamento della stima conservata da un dato nodo:

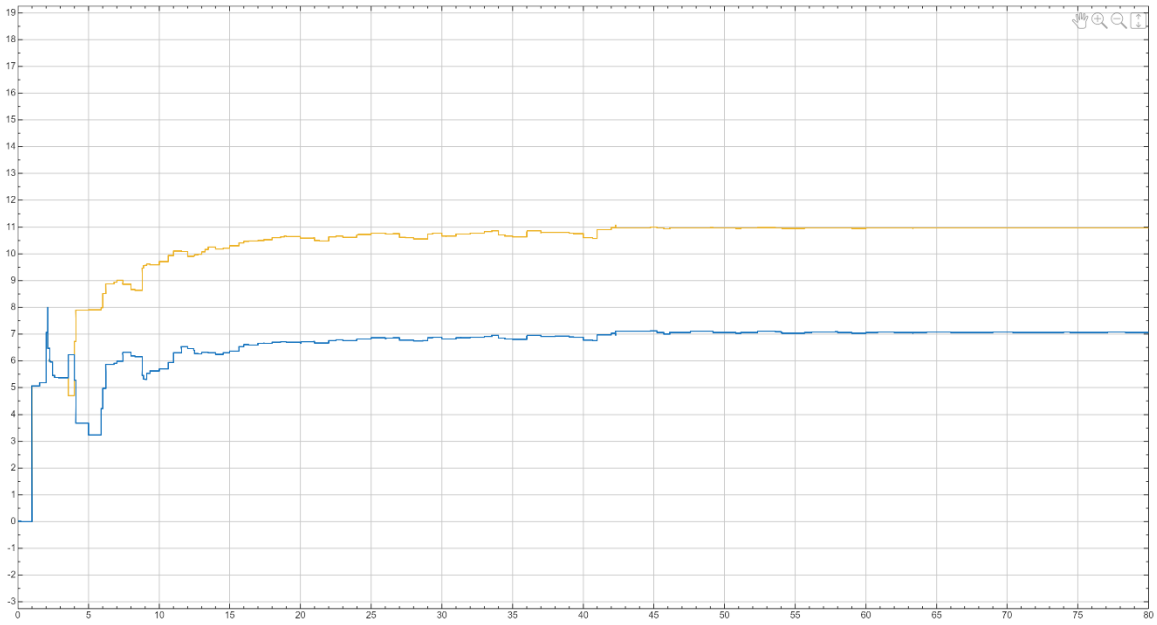


**Figure 30.** Media di un nodo

Vediamo quindi che da un certo istante in poi (da 40 in poi) la posizione si assesta ad un valore comune a tutti i nodi.

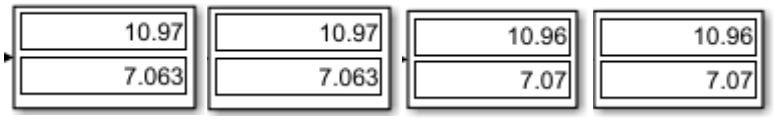



**Figure 31.** Output di ogni nodo

A meno di un errore trascurabile, tutti convergono alla medesima posizione dell'oggetto.

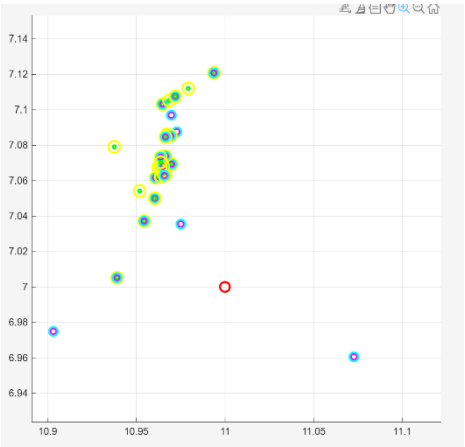



**Figure 32.** Zoom sul plot

Vediamo graficamente che tutti decido per lo stesso punto.

### 3.10. Livello energetico

Per concludere la parte simulativa introduciamo anche il livello energetico, assumiamo il seguente modello:

$$\dot{E} = -E_{idle/comp} - E_{TX}$$
$$E(0) = 100 \tag{6}$$

Il nodo **inizialmente** ha una energia di **100**, la quale nel tempo può solo diminuire. Si sta quindi simulando un nodo dotato di una fonte di energia limitata, come una batteria. Le causa della scarica sono due:

- Energia di idle e computazione: necessaria per il funzionamento del nodo.
- Energia di trasmissione, quando il nodo deve trasmettere le informazioni ai vicini il consumo aumenta, a causa all'utilizzo del modulo di trasmissione.

In simulink ciò è stato implementato nel seguente modo:

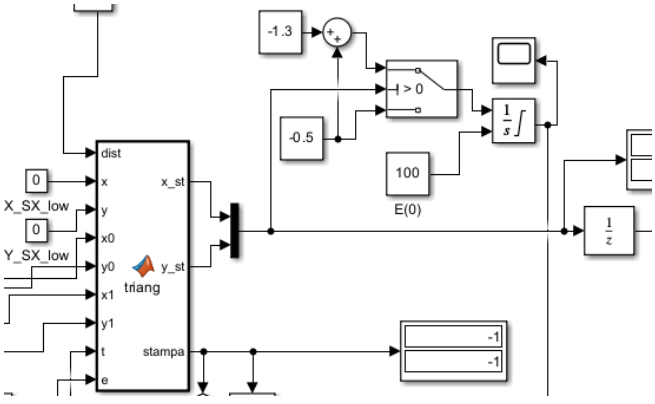



**Figure 33.** Livello energetico

Si considera l'uscita del nodo, se questa è pari a -1 ricordiamo che il nodo non sta trasmettendo e assumiamo che il consumo sia di 0.5, mentre se l'uscita è diversa da -1 significa che sta trasmettendo quindi al consumo di 0.5 si somma un consumo di 1.3. Ciò va in ingresso ad un blocco integratore che permette di implementare la (6) cosi come accade anche nel modello della coda della rete. Infine, il valore di energia è dato in ingresso alla matlab function, in modo tale che se il valore di energia è superiore a zero, il nodo funzioni normalmente, mentre se la batteria si scarica il nodo si "spegne". Ogni nodo viene così ampliato, si evita di riportare nuovamente l'intera architettura. Per vedere l'effetto di ciò simuliamo un oggetto che viene visto solo da alcuni nodi:

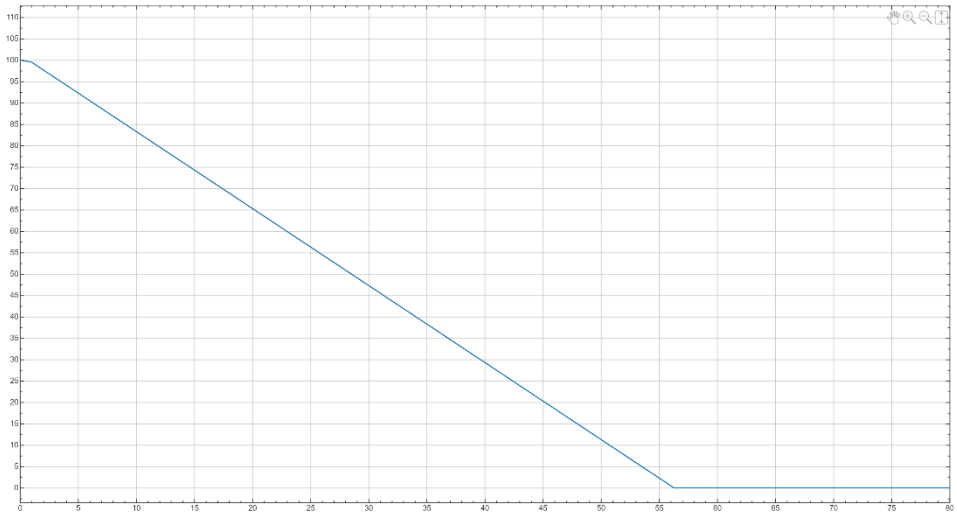


**Figure 34.** Scarica di un nodo che trasmette sempre

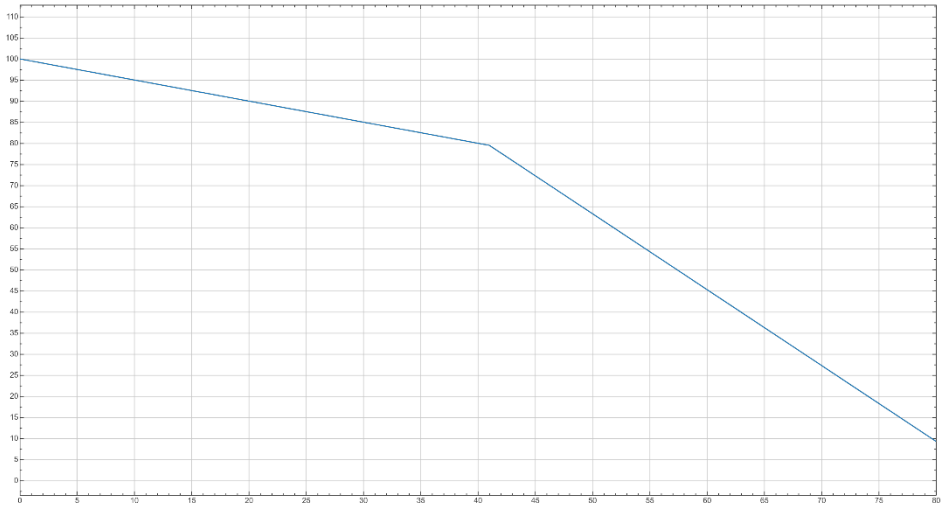


**Figure 35.** Scarica di un nodo che inizialmente non trasmette

Come notiamo dalle immagini riportate il primo nodo vede sempre l'oggetto per questo trasmette sempre, consumando più velocemente la propria batteria. Mentre in figura 25, vediamo l'andamento dell'energia di un nodo che non vede l'oggetto. Perciò nella prima fase di stima distribuita riceve solo, consumando poca energia, mentre nella seconda fase di media, partecipando tutti allora trasmette anche lui consumando più energia. Inoltre, vediamo che il primo nodo ad un certo punto si scarica e smetterà di partecipare, come possiamo vedere anche dalla propria conoscenza:

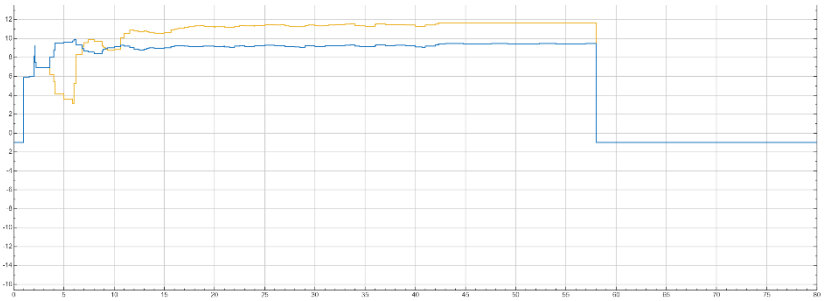


**Figure 36.** Spegnimento di un nodo.

## 4. Implementazione fisica

Dimostrato mediante simulazione il corretto funzionamento dell'algoritmo possiamo ora implementarlo in un sistema fisico e reale, per fare ciò sono stati realizzati 4 nodi fisici, degli ESP8266. Ogni nodo inoltre è dotato di sensore di prossimità HC-SR04 secondo il seguente **schema circuitale**:

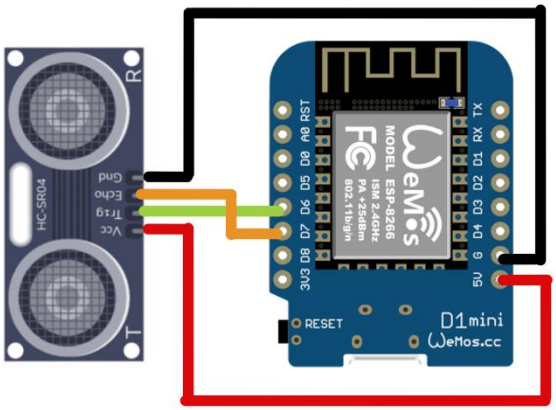



**Figure 37.** Schema del nodo

Ogni nodo è stato posizionato su una torretta posta agli angoli di un'area da sorvegliare così come mostrato inizialmente e in simulazione.

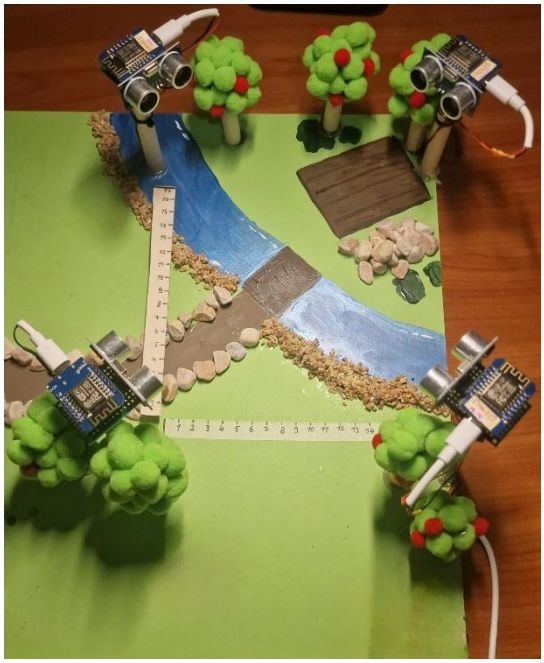


**Figure 38.** Nodi nella realtà

### 4.1. Comunicazione mesh

Per comunicare i nodi utilizzano il protocollo **PainlessMesh**, con il quale è possibile realizzare in maniera estremamente facile una rete di nodi. Per fare ciò è sufficiente inizializzare il protocollo e definire le funzioni di callback:

```
mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT, WIFI_AP_STA, 6);
mesh.onReceive(&receivedCallback);
mesh.onNewConnection(&newConnectionCallback);
mesh.onChangedConnections(&changedConnectionCallback);
mesh.onNodeTimeAdjusted(&nodeTimeAdjustedCallback);
```

Le ultime 4 istruzioni servono a definire quali funzioni eseguire quando si verifica un dato evento. Quella che maggiormente ci interessa è la callback eseguita a seguito della **ricezione di un messaggio**; infatti, con il contenuto di esso applicheremo l'algoritmo.

Mentre per quanto riguarda l'invio dei messaggi si usa la seguente funzione:

```
String msg = "1-"+String(x_st)+"-"+String(y_st);
mesh.sendBroadcast(msg);
```

La struttura dei messaggi inviati è, come si vede dal codice:

| type | Stima di X | Stima di Y |
|---|---|---|

Notiamo la presenza di un campo type, il quale è necessario poiché ogni nodo oltre che inviare i dati ai vicini, gli invia anche ad un nodo **sniffer**, tale nodo non partecipa all'algoritmo, ma ha il solo scopo di fornire i dati ad un programma python per poterli stampare in maniera analoga a matlab, come vedremo nei seguenti test.

### 4.2. Codice

Riportiamo di seguito il codice eseguito da ogni nodo, ovviamente alcune piccole parti come nome e posizione del nodo non sono uguali per tutti. Si vuole però sottolineare due aspetti del codice riportato ne seguito:

- Tale codice non implementa la seconda fase in cui avviene la media distribuita, nonostante essa fosse facilmente implementabile e funzionante. Il problema nasce nel volerla usare come fase due. A differenza di matlab in cui i nodi passavano in maniera praticamente simultanea da una fase alla successiva, nella realtà ciò non accade. Perciò, si verificavano situazioni in cui alcuni nodi utilizzavano i dati per una fase mentre altri per la successiva, rendendo completamente errato il risultato.
  - Una prima soluzione ideata consisteva nel fornire un segnale esterno che facesse passare da una fase all'altra. Ovviamente ciò non rispecchia i fini del progetto in quanto rendeva il tutto più centralizzato.
  - Perciò la seconda soluzione è stata rendere un nodo leader, il quale mediante un altro algoritmo distribuito facesse in modo che tutti passassero a tale fase, poiché come visto a lezione tale algoritmo, algoritmo del consenso con leader/riferimento, è molto simile a quanto già implementato e usato nel seguito per l'intervento dei dronoi, si è evitato di appesantire ulteriormente il tutto.
- Anche nell'implementazione fisica il livello energetico è stato simulato, a causa di varie complicazioni discusse a lezione. Perciò si usa una variabile energia, la quale viene decrementata ogni qual volta si effettua una trasmissione. Quando il nodo raggiunge energia 0 si spegne.

```
//importiamo le librerie necessarie
#include "painlessMesh.h"
#include "math.h"
//definiamo i parametri della rete
#define MESH_PREFIX   "MeshNetwork"
#define MESH_PASSWORD "password123"  // minimo 8 caratteri
#define MESH_PORT     5555

Scheduler userScheduler;
painlessMesh mesh;

//pin del sensore di prossimità
#define trigPin D6
#define echoPin D7

//variabili che conservano la posizione stimata
float x_st=0.1;
float y_st=0.1;

//POSIZIONE  E NOME DEL NODO
String nome="Pippi";
float x=0;
float y=17;
```

```
//Nodo che si chiama pippi parla solo con 2 vicini: vuoto e alessia
//di ID: 3958302360 (vuoto) 5702239 (alessia)
uint32_t vicino1=3958302360;
uint32_t vicino2=5702239;

//**********
float durata, cm;
bool led=false;

//carica della batteria
int energia=200;

//strutture dati di appoggio per contenere quanto ricevuto dai vicini
struct stima_vicino {
    uint32_t id;
    float x;
    float y;
};
stima_vicino st_vicini[3] = {
    {0, 0.0, 0.0},   // 0 = slot libero
    {0, 0.0, 0.0},
    {0, 0.0, 0.0}
};
//funzione ausiliaria per la gestione delle strutture precedenti
void inserisci(uint32_t id, float x, float y) {
    // 1) Cerca se l'ID esiste già
    for (int i = 0; i < 3; i++) {
        if (st_vicini[i].id == id) {
            st_vicini[i].x = x;   // aggiorna
            st_vicini[i].y = y;
            return;
        }
    }
    // 2) Altrimenti aggiungi in coda (primo slot libero)
    for (int i = 0; i < 3; i++) {
        if (st_vicini[i].id == 0) {         // slot libero
            st_vicini[i].id = id;
            st_vicini[i].x = x;
            st_vicini[i].y = y;
            return;
        }
    }
    // 3) Se arrivi qui, il vettore è pieno
    Serial.println("Vettore pieno, impossibile inserire");
}

// Task: ogni secondo effettua una misura
Task taskMisura(TASK_SECOND * 1, TASK_FOREVER, []() {
    //ogni volta che misuriamo lampeggiamo il led
    if (energia > 0){
        digitalWrite(LED_BUILTIN, led);
            led=!led;
        }else{
            digitalWrite(LED_BUILTIN, LOW);
        }

    //effettuiamo la misura vera e propria
    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);
    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
```

```
    digitalWrite(trigPin, LOW);
    durata = pulseIn(echoPin, HIGH);
    cm = durata / 58;
    Serial.print("Cm = ");
    Serial.print(cm);
    Serial.print("Mia stima: ");
    Serial.print(x_st);
    Serial.print(", ");
    Serial.println(y_st);

    //ottenuta la misura verifico se ci sia l'intruso
    if(cm<=20){
        //incaso rilevo qualcosa, invio la mia stima ai vicini
        String msg ="1-"+String(x_st)+"-"+String(y_st);
        mesh.sendBroadcast(msg);
        //avendo effettuato una trasmissione decremento l'energia
        energia--;
        Serial.print("RILEVATO invio e decremento");
        Serial.println (energia);
    }
});

//ogni qual volta ricevo un messaggio dai vicini eseguo tale funzione
//la quale esegue l'algoritmo mostrato nei paragrafi precedenti
void stimadistribuita(){
    float x_medio;
    float y_medio;

    int n=1;

    x_medio=x_st;
    y_medio=y_st;

    for(int i=0; i<3; i++){
        if(st_vicini[i].id==vicino1 || st_vicini[i].id==vicino2){
            x_medio+=st_vicini[i].x;
            y_medio+=st_vicini[i].y;
            n++;
        }
    }

    x_medio=x_medio/n;
    y_medio=y_medio/n;

    //come gia mostrato se io vedo il nodo allora eseguo l'algoritmo
descritto
    if (cm<=20){
        float rx = x_medio-x;
        float ry = y_medio-y;
        float r=sqrt(rx*rx+ry*ry);
        float error=r-cm;

        float ux=rx/r;
        float uy=ry/r;

        x_st = x_medio - 0.2 * error * ux;
        y_st = y_medio - 0.2 * error * uy;
        Serial.println("Vedo");
    }else{
        //ma se ricevo messaggi ma non vedo allora faccio banalmente una
media
```

```
        x_st = x_medio;
        y_st = y_medio;
        Serial.println("Non vedo");
    }


    //stampa di debug serve solo per lo sniffer per fare il grafico
    String msg = "2-"+String(x_st)+"-"+String(y_st);
    mesh.sendBroadcast(msg);

    Serial.print("nuova stima calcolata: ");
    Serial.print(x_st);
    Serial.print(",");
    Serial.println(y_st);
}

//callBack eseguita quando si riceve un messaggio
void receivedCallback(uint32_t from, String &msg) {
    //si estrae il messaggio
    //Serial.printf("[RX] Da %u: %s| ", from, msg.c_str());
    int sep1 = msg.indexOf('-');
    int sep2 = msg.indexOf('-', sep1 + 1);

    float type = msg.substring(0, sep1).toFloat();
    float a = msg.substring(sep1 + 1, sep2).toFloat();
    float b = msg.substring(sep2 + 1).toFloat();

    //se è un messaggio dell'algoritmo e non di debug allora chiamiamo la
funzione precedente
    //ovviamente solo se abbiamo ancora energia
    if(type==1 && energia>0){
        inserisci(from,a,b);
        stimadistribuita();
    }

}

void newConnectionCallback(uint32_t nodeId) {
    Serial.printf("[+] Nuovo nodo connesso: %u\n", nodeId);
    Serial.printf("    Nodi in rete: %d\n", mesh.getNodeList().size() +
1);
}

void changedConnectionCallback() {
    Serial.printf("[~] Topologia cambiata: %s\n",
mesh.subConnectionJson().c_str());
}

void nodeTimeAdjustedCallback(int32_t offset) {
    Serial.printf("[T] Clock sincronizzato, offset: %d us\n", offset);
}

void setup() {
    Serial.begin(115200);
    delay(200);

    //mesh.setDebugMsgTypes(ERROR | STARTUP | CONNECTION);
    mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT,
WIFI_AP_STA, 6);
    mesh.onReceive(&receivedCallback);
    mesh.onNewConnection(&newConnectionCallback);
```

```
    mesh.onChangedConnections(&changedConnectionCallback);
    mesh.onNodeTimeAdjusted(&nodeTimeAdjustedCallback);

    pinMode(trigPin, OUTPUT);
    pinMode(echoPin, INPUT);
    pinMode(LED_BUILTIN, OUTPUT);

    delay(2000);
    userScheduler.addTask(taskMisura);
    taskMisura.enable();

    Serial.printf("\n=== NODO AVVIATO ===\n");
    Serial.printf("ID:      %u\n", mesh.getNodeId());
    Serial.printf("NOME:  %s\n",nome);
    Serial.print("Sono in angolo: ");
    Serial.print(x);
    Serial.print(",");
    Serial.println(y);
    Serial.println("====================\n");
}

void loop() {
    mesh.update();
    if(energia<=0){
        Serial.print("scaricato");
        Serial.println (energia);
            digitalWrite(LED_BUILTIN, 1);
            taskMisura.disable();
    }
}
```

### 4.3. Test

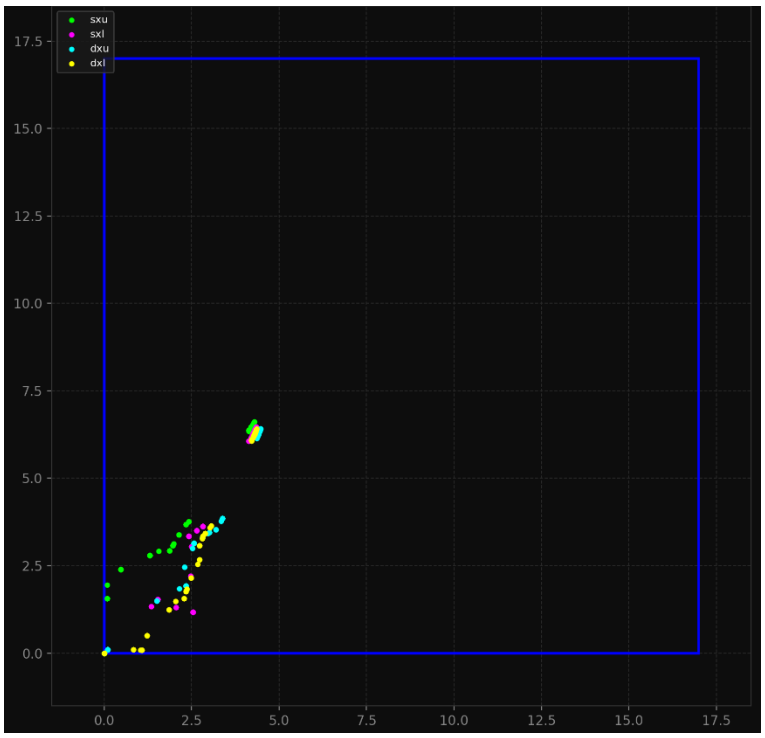



**Figure 39.** Test con punto di partenza (0,0)

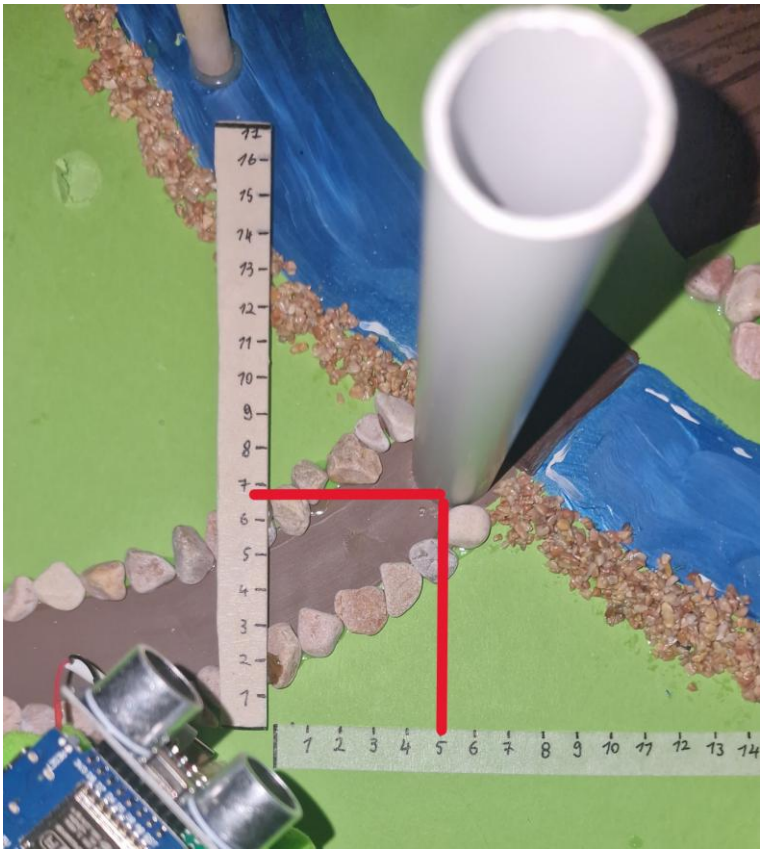


**Figure 40.**

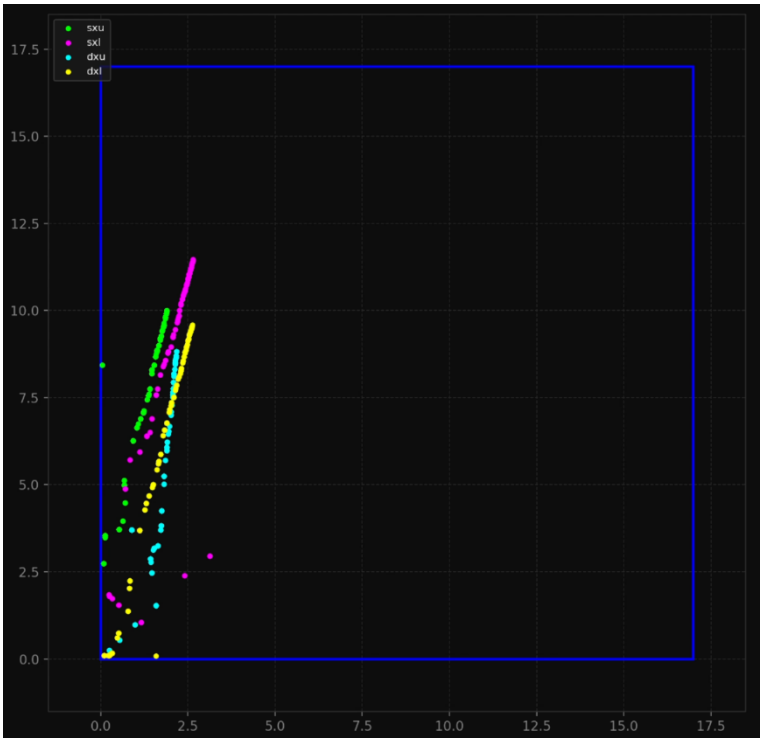



**Figure 41.** Test con punto di partenza (0,0)

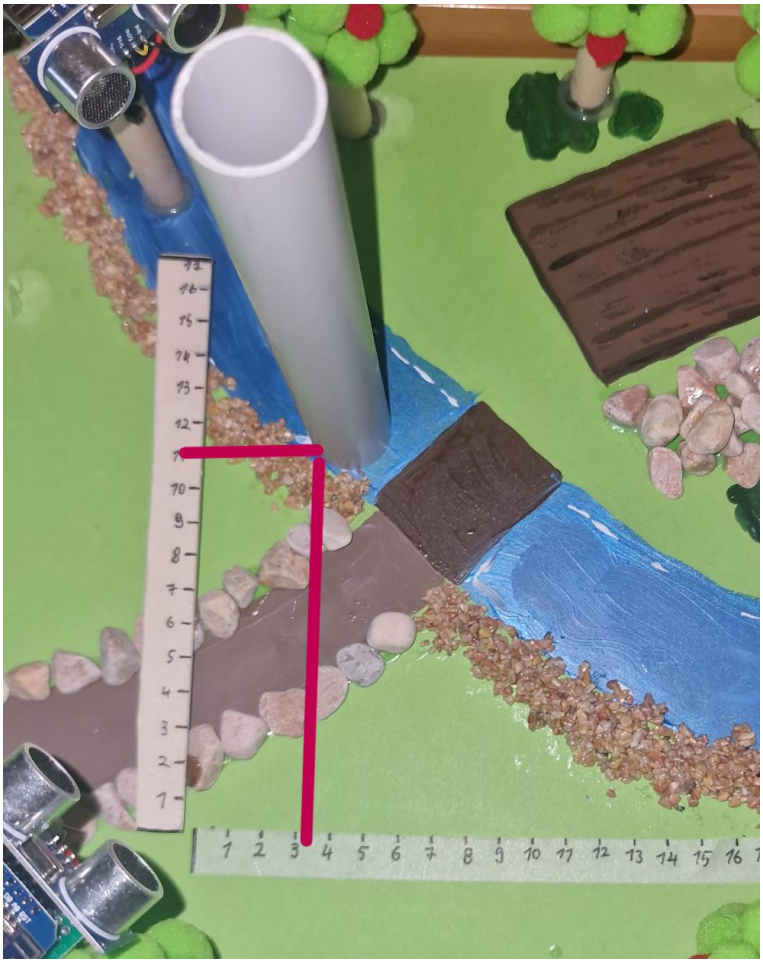


**Figure 42.**

Evidenziamo che nella pratica, usare il punto di partenza (0,0) non è sempre l'ideale. Perciò alcuni test sono stati effettuati con punto di partenza centrale (8.5,8.5)

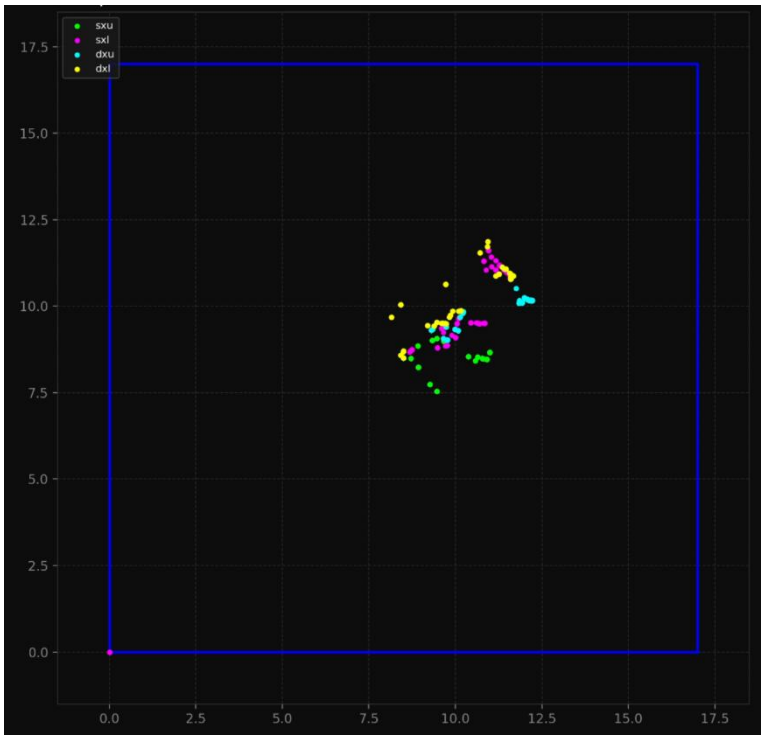



**Figure 43.** Test con punto di partenza (8.5,8.5)

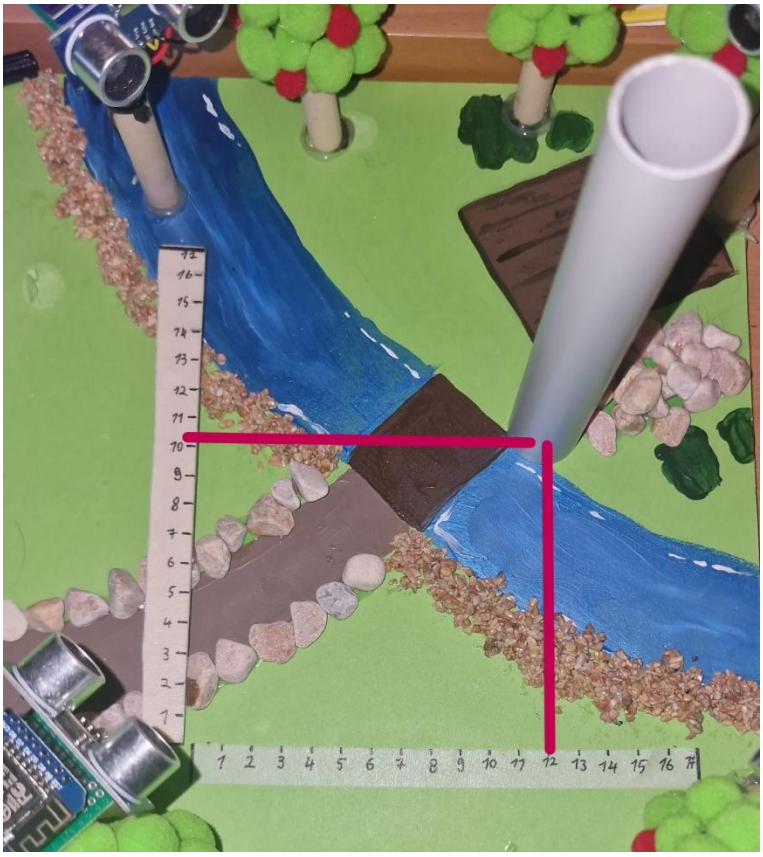


**Figure 44.**

## 5. Intervento dei droni.

Vogliamo ora simulare l'intervento di una **flotta di droni**, i quali si dovranno recare nella zona in cui è stato localizzato l'intruso. Per fare ciò quindi i droni utilizzeranno la posizione stimata dagli altri nodi. Nel nostro esempio consideriamo due droni di cui però solo uno comunica con il resto della rete e conosce la posizione stimata, ciò viene considerato come un riferimento esterno. Perciò ogni nodo userà **l'algoritmo del consenso con riferimento/leader** per convergere alla posizione target. La rete è così modificata:

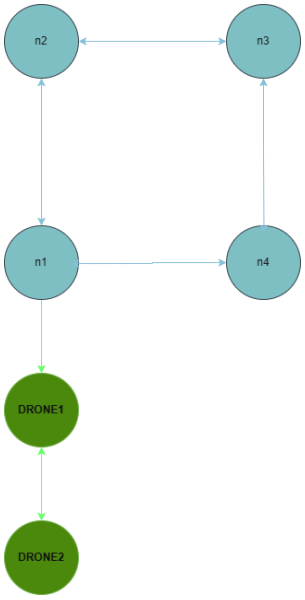



**Figure 45.** Rete con droni.

I nodi droni sono composti da **tre parti** principali:

- Una matlab function che implementa l'algoritmo del consenso con riferimento/leader, permettendo a tutti i droni di conoscere la posizione target.
- Un controllore PI, il quale comanda il drone per portarlo nella posizione target ottenuta dal blocco precedente.
- Due processi del secondo ordine che modellano il comportamento del drone l'ungo l'asse x e lungo l'asse y.

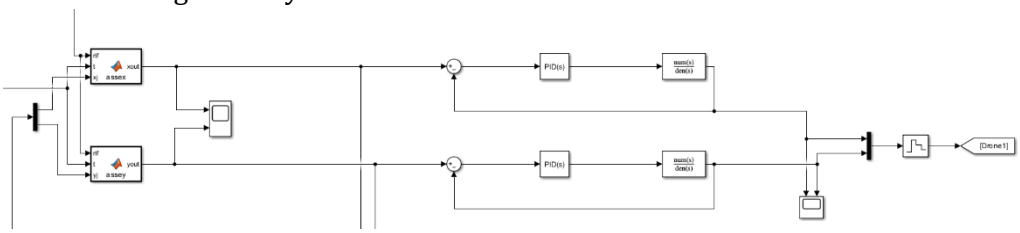



**Figure 46.** Architettura drone1

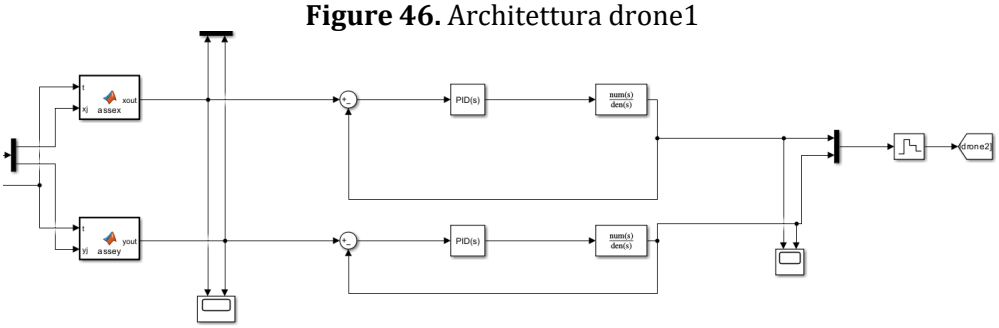



**Figure 47.** Architettura drone2

I due droni, per poter applicare l'algoritmo, si scambiano la propria conoscenza locale, ovvero la loro stima della posizione target. Ma come possiamo vedere solo il drone1 riceve il riferimento dalla restante parte della rete, mentre il drone2 scambia solo la propria conoscenza. Così facendo entrambi convergono alla stessa posizione di riferimento. Ciò è fornito anche al resto del nodo drone, viene calcolato l'errore sottraendo alla posizione target la posizione reale del drone, l'errore cosi calcolato viene fornito al controllore PI che controlla il processo fisico del secondo ordine che emula il comportamento del drone (ciò è fatto sia per l'asse x che per l'asse y). L'implementazione dell'algoritmo nel primo drone è la seguente:

```
function xout  = assex(rif, t, xj)

    persistent x
    persistent riferimento;

    if isempty(x) && isempty(riferimento)
        x=0;
        riferimento=0;
    end

    if t==80
        riferimento=rif(1);
    end

    if t>80
        x=x-1*0.5*(x-xj)-1*0.5*(x-riferimento);
        xout=x;
    else
        xout=0;
    end

end
```

NOTA: in maniera analoga è implementata per l'asse y.

Mentre l'implementazione dell'algoritmo nel secondo drone, che ricordiamo non conosce il riferimento, è:

```
function xout  = assex(t, xj)

    persistent x

    if isempty(x)
        x=0;
    end

    if t>80
        x=x-1*0.5*(x-xj);
        xout=x;
    else
        xout=0;
    end
end
```

NOTA: in maniera analoga è implementata per l'asse y.
Anche nel caso della comunicazione fra i due nodi è stata considerata la presenza della rete per emulare le varie non idealità citate in precedenza, perciò l'architettura dei due droni è la seguente:

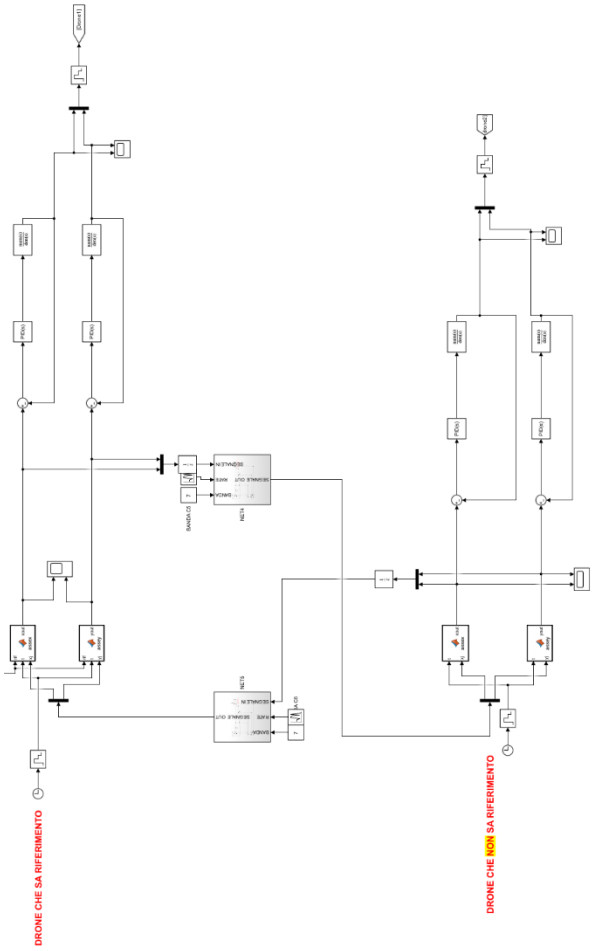



**Figure 48.** Architettura con due droni

Per simulare il movimento del drone verso la posizione stimata, si è modellato ciascun asse (X e Y) come un sistema lineare del secondo ordine, descritto dalla seguente funzione di trasferimento:

$$\frac{1}{(s^2 + 2 * \varphi * \omega_n * s + \omega_n^2)}$$

dove $\omega_n$rappresenta la frequenza naturale del sistema, che determina la velocità di risposta, e $\zeta$è il coefficiente di smorzamento, che ne governa il comportamento transitorio. Scegliamo $\omega_n$=0.5 per una risposta volutamente lenta, coerente con le dinamiche di un drone che deve raggiungere con gradualità la posizione e $\varphi = 0.8$ per far si che il drone converge alla posizione stimata senza oscillazioni eccessive:

$$\frac{0.25}{(s^2 + 0.8s + 0.25)}$$

Perciò l'architettura complessiva risulta essere:

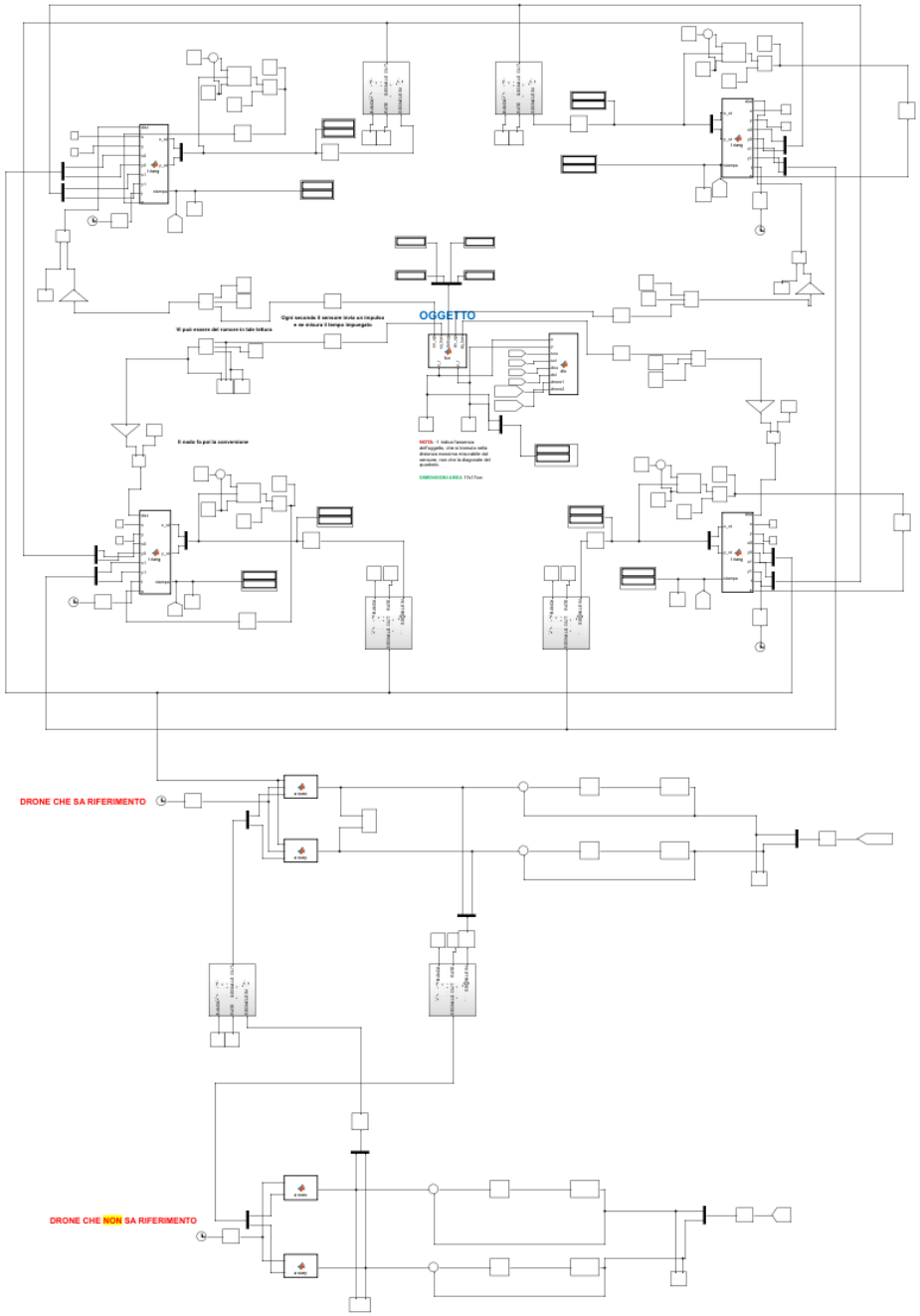



**Figure 49.** Architettura finale

### 5.1. Simulazione

Assumiamo conclusa la stima della posizione dell'intruso da parte dei 4 nodi con sensori, come già visto il risultato era il seguente:

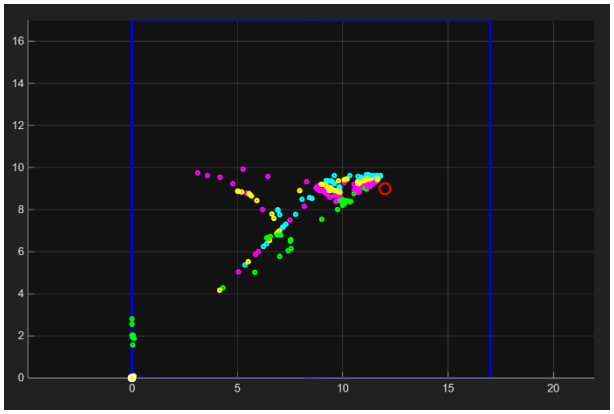


**Figure 50.**

Ciò è fornito in input solo al drone1, il quale usa ciò come riferimento per applicare l'algoritmo del consenso con riferimento/leader, per far si che sia noto anche al drone 2.

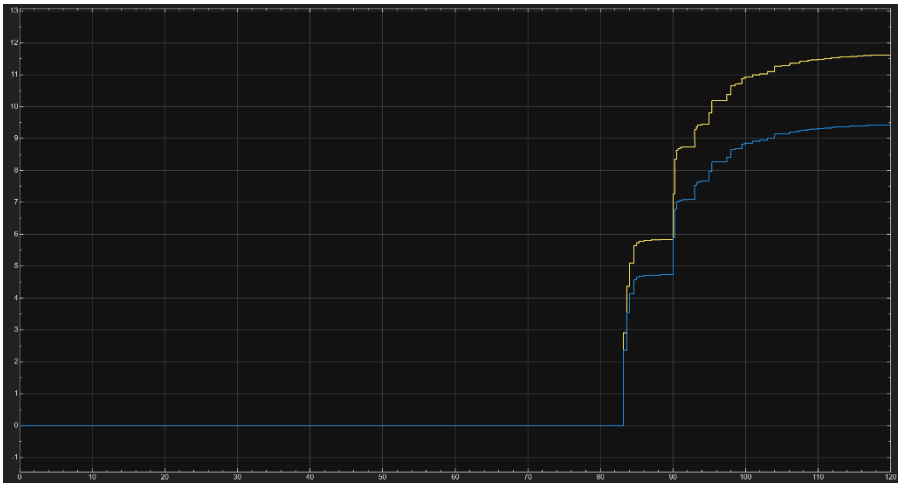


**Figure 51.** Conoscenza del drone 2

Come possiamo vedere inizialmente il drone ha conoscenza nulla poiché la fase di stima non è terminata, ma non appena viene eseguito l'algoritmo citato la sua conoscenza converge alla posizione target.

Perciò nota la destinazione il controllore PID porta i droni in tale posizione.

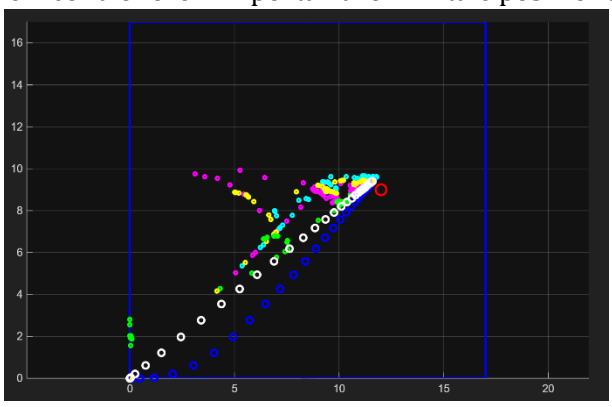


**Figure 52.** Intervento dei droni

Quello in blu è il drone a cui è noto il riferimento, mentre quello bianco è quello a cui non è noto.